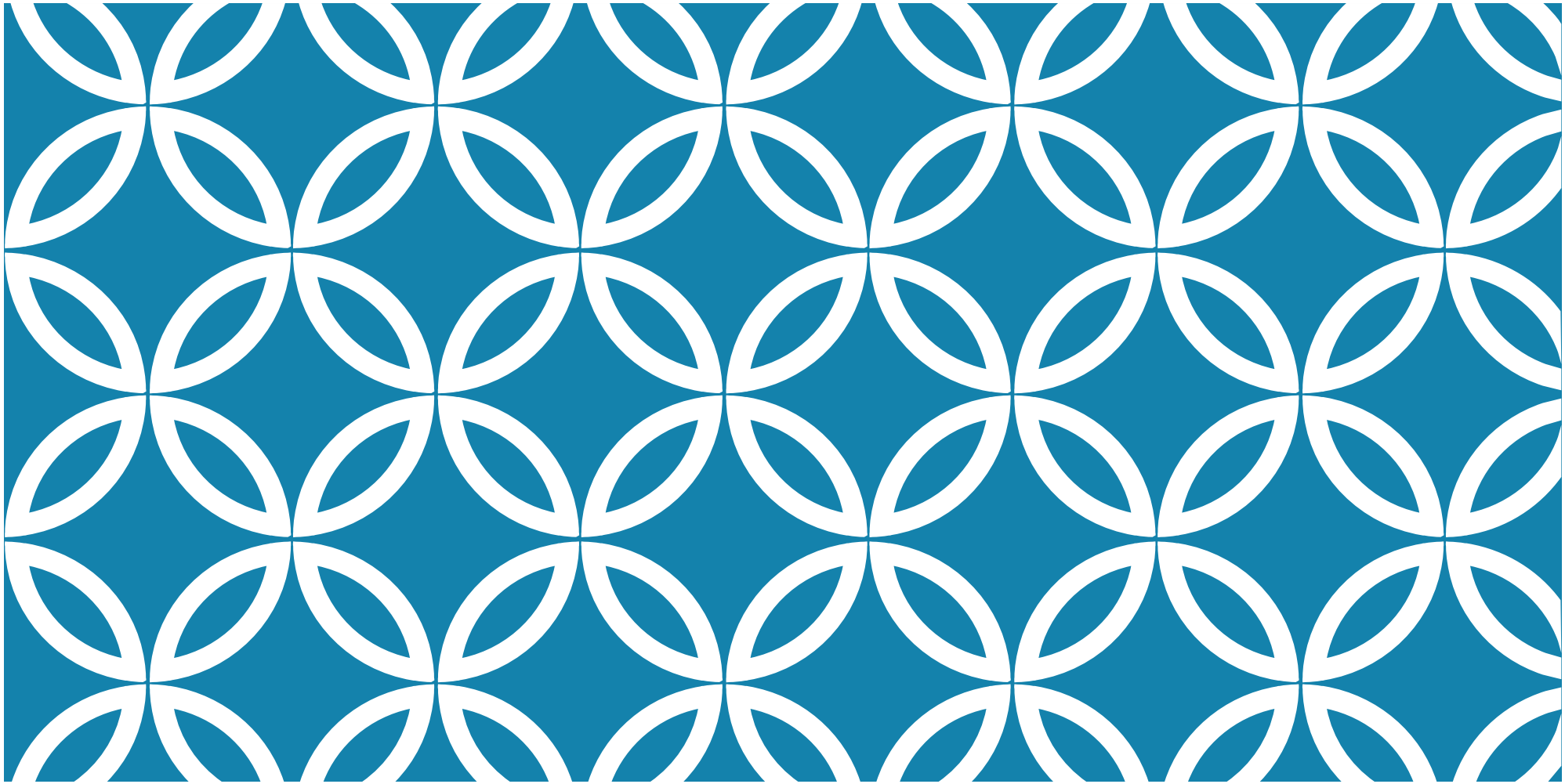
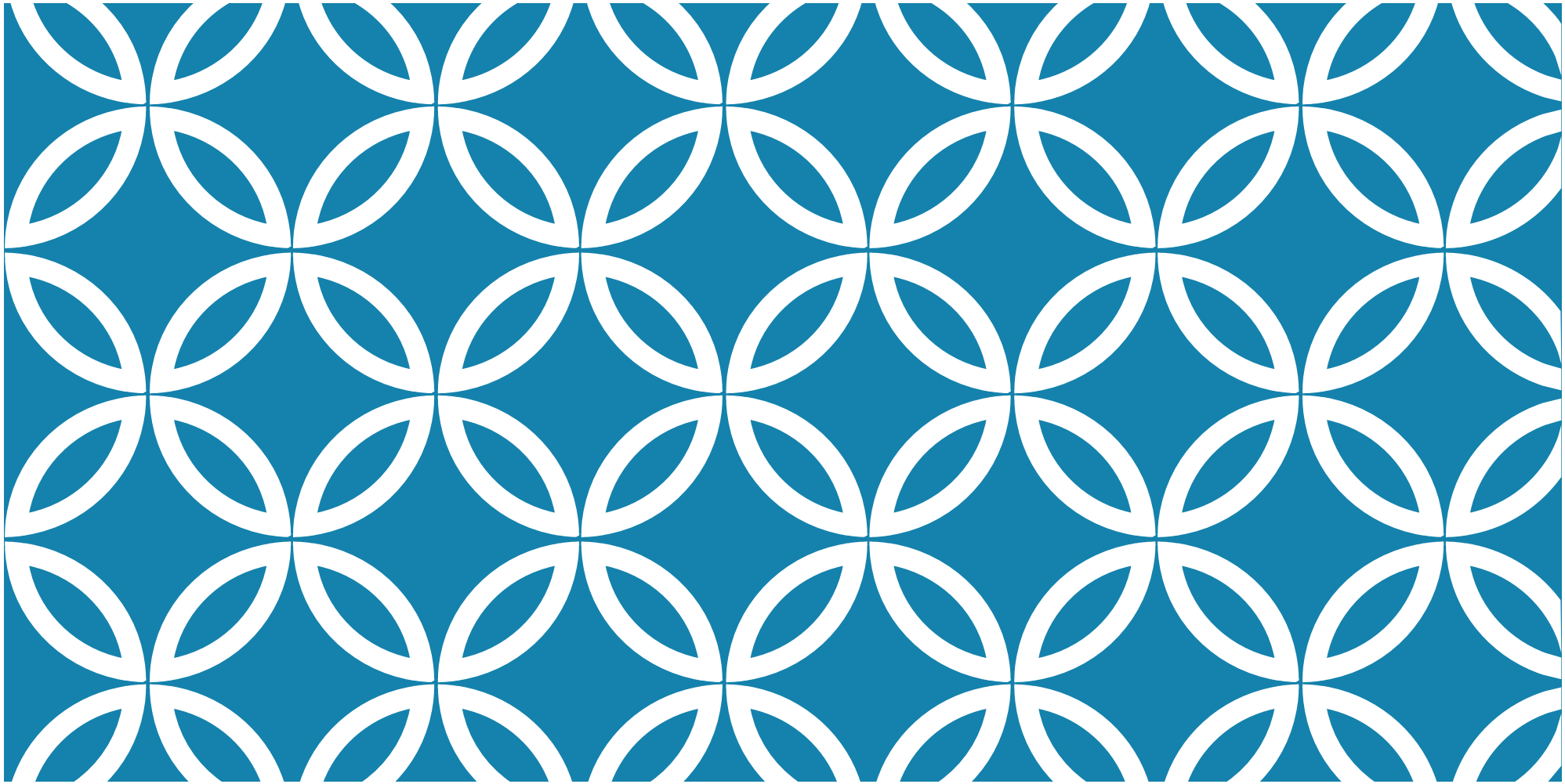


DEVELOPPEMENT API REST

Par : Evaris Fomekong

Année : 2026



LECTURE 1 : FONDAMENTAUX REST ET ARCHITECTURE API-FIRST AVEC OPENAPI

- Principes architecturaux REST : ressources, URIs, représentations et contraintes de l'architecture RESTful (stateless, cacheable, layered system), - Méthodes HTTP et leur sémantique : GET, POST, PUT

INTRODUCTION AUX FONDAMENTAUX REST (PARTIE 1/2)

REST (Representational State Transfer) est un style architectural défini par Roy Fielding en 2000 pour concevoir des systèmes distribués sur le web. Il repose sur un ensemble de contraintes architecturales qui garantissent la scalabilité, la simplicité et la performance des services web. REST exploite les standards HTTP existants pour créer des APIs intuitives et interopérables.

INTRODUCTION AUX FONDAMENTAUX REST (PARTIE 2/2)

Objectifs pédagogiques de cette section :

Comprendre les principes architecturaux fondamentaux de REST

Maîtriser les concepts de ressources, URIs et représentations

Connaître les contraintes architecturales RESTful

Utiliser correctement les méthodes HTTP et codes de statut

Adopter l'approche API-First dans le cycle de développement

CONCEPT DE RESSOURCE : FONDATION DE REST (PARTIE 1/3)

Dans l'architecture REST, une ressource est toute information pouvant être nommée : un document, une image, un service, une collection d'autres ressources, ou un objet métier. Chaque ressource est identifiée de manière unique par un URI (Uniform Resource Identifier). Le concept clé est que le client manipule des représentations de ressources, jamais directement les ressources elles-mêmes.

CONCEPT DE RESSOURCE : FONDATION DE REST (PARTIE 2/3)

Exemples d'URIs RESTful pour différents types de ressources (http)

```
# Exemples d'URIs pour différentes ressources

# Collection de ressources
GET https://api.exemple.com/users

# Ressource individuelle
GET https://api.exemple.com/users/42

# Sous-ressource
GET https://api.exemple.com/users/42/orders

# Ressource spécifique dans une sous-collection
GET https://api.exemple.com/users/42/orders/1337

# Ressource avec filtrage (query parameters)
GET https://api.exemple.com/products?category=electronics&price_max=500
```

CONCEPT DE RESSOURCE : FONDATION DE REST (PARTIE 3/3)

Bonnes pratiques pour nommer les ressources :

Utiliser des noms au pluriel pour les collections (users, products, orders)

Privilégier les noms plutôt que les verbes dans l'URI

Utiliser des hiérarchies pour exprimer les relations (/users/42/orders)

Être cohérent dans la convention de nommage (kebab-case ou snake_case)

Éviter les URIs trop profonds (maximum 3-4 niveaux recommandés)

REPRÉSENTATIONS DE RESSOURCES (PARTIE 1/3)

Une représentation est une forme concrète des données d'une ressource à un instant donné. Elle capture l'état actuel de la ressource dans un format spécifique (JSON, XML, HTML, etc.). Le serveur envoie des représentations au client, et le client peut envoyer des représentations au serveur pour modifier l'état des ressources. La négociation de contenu permet au client et au serveur de s'accorder sur le format optimal.

REPRÉSENTATIONS DE RESSOURCES (PARTIE 2/3)

Représentation JSON d'une ressource utilisateur avec HATEOAS (json)

```
1. // Exemple de représentation JSON d'une ressource User
2. {
3.   "id": 42,
4.   "username": "jean.dupont",
5.   "email": "jean.dupont@example.com",
6.   "firstName": "Jean",
7.   "lastName": "Dupont",
8.   "createdAt": "2024-01-15T10:30:00Z",
9.   "isActive": true,
10.   "roles": ["user", "contributor"],
11.   "_links": {
12.     "self": "/users/42",
13.     "orders": "/users/42/orders",
14.     "profile": "/users/42/profile"
15.   }
16. }
```

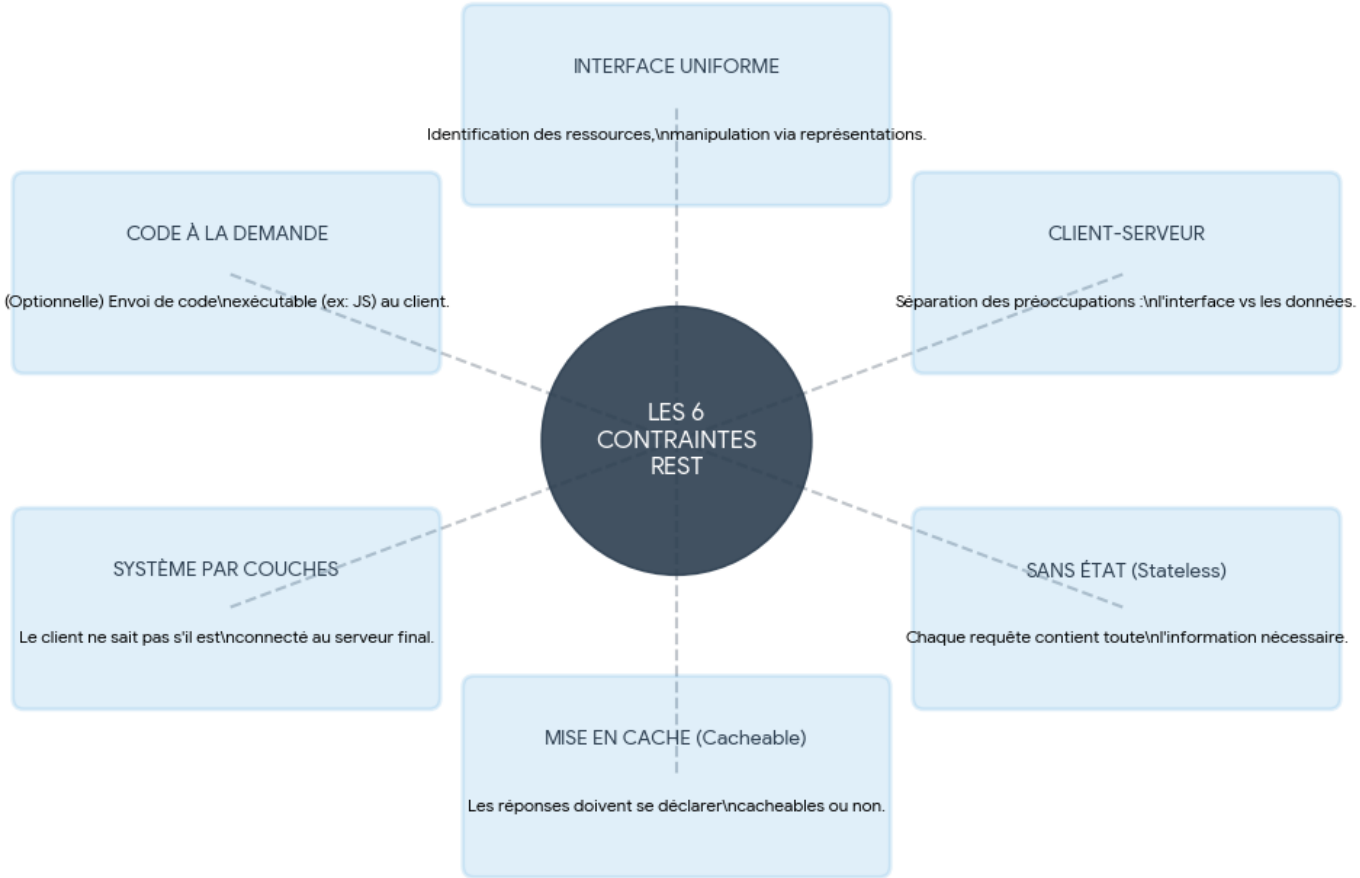
REPRÉSENTATIONS DE RESSOURCES (PARTIE 3/3)

Négociation de contenu avec les headers Accept et Content-Type (http)

```
# Négociation de contenu via les headers HTTP

# Client demande une représentation JSON
GET /users/42 HTTP/1.1
Host: api.exemple.com
Accept: application/json

# Client demande une représentation XML
GET /users/42 HTTP/1.1
Host: api.exemple.com
Accept: application/xml

# Réponse du serveur indiquant le format
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 324
```

LES 6 CONTRAINTES ARCHITECTURALES REST (PARTIE 1/2)

Diagramme des 6 Contraintes Architecturales REST



LES 6 CONTRAINTES ARCHITECTURALES REST (PARTIE 2/2)

Client-Server : Séparation des préoccupations entre interface utilisateur et stockage de données

Stateless : Chaque requête contient toutes les informations nécessaires, aucun contexte stocké sur le serveur

Cacheable : Les réponses doivent explicitement indiquer si elles peuvent être mises en cache

Uniform Interface : Interface uniforme entre composants (identification des ressources, manipulation via représentations)

Layered System : Architecture en couches où chaque composant ne voit que la couche adjacente

Code-On-Demand (optionnel) : Le serveur peut étendre les fonctionnalités du client en transférant du code exécutable

CONTRAINTE STATELESS : IMPLICATIONS PRATIQUES (PARTIE 1/3)

La contrainte stateless est fondamentale en REST : le serveur ne conserve aucun état de session client entre les requêtes. Chaque requête doit contenir toutes les informations nécessaires pour être comprise et traitée. Cette contrainte améliore la scalabilité (pas de synchronisation d'état entre serveurs), la fiabilité (pas de perte d'état en cas de panne) et la visibilité (chaque requête est autonome).

CONTRAINTE STATELESS : IMPLICATIONS PRATIQUES (PARTIE 2/3)

Comparaison entre approche stateful et stateless (http)

```
# ❌ MAUVAISE PRATIQUE : API avec état (stateful)
# Requête 1 : Connexion
POST /login
{"username": "jean", "password": "secret"}
# Serveur stocke la session

# Requête 2 : Récupération de profil (dépend de la session)
GET /profile
# Le serveur utilise la session stockée

# ✅ BONNE PRATIQUE : API sans état (stateless)
# Requête 1 : Authentification
POST /auth/token
{"username": "jean", "password": "secret"}
# Réponse : {"access_token": "eyJhbGc..."}

# Requête 2 : Récupération de profil avec token
GET /users/42/profile
Authorization: Bearer eyJhbGc...
# Chaque requête contient toutes les informations nécessaires
```

CONTRAINTE STATELESS : IMPLICATIONS PRATIQUES (PARTIE 3/3)

Avantages de l'approche stateless :

Scalabilité horizontale facilitée (load balancing simple)

Résilience accrue (pas de perte d'état en cas de crash serveur)

Mise en cache simplifiée

Débogage plus facile (requêtes auto-suffisantes)

Défis de l'approche stateless :

Taille des requêtes potentiellement plus importante

Nécessité de tokens d'authentification (JWT, OAuth)

Complexité transférée au client

CONTRAINTE CACHEABLE : OPTIMISATION DES PERFORMANCES (PARTIE 1/3)

La contrainte cacheable impose que les réponses doivent explicitement indiquer si elles peuvent être mises en cache ou non. Cela permet d'éliminer partiellement ou complètement certaines interactions client-serveur, améliorant ainsi l'efficacité, la scalabilité et les performances perçues par l'utilisateur. Les headers HTTP Cache-Control, ETag, Expires et Last-Modified sont les mécanismes standards.

CONTRAINTE CACHEABLE : OPTIMISATION DES PERFORMANCES (PARTIE 2/3)

Utilisation des headers de cache et requêtes conditionnelles (http)

```
# Exemple de réponse avec directives de cache

HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: public, max-age=3600
ETag: "33a64df551425fcc55e4d42a148795d9f25f89d4"
Last-Modified: Wed, 15 Jan 2024 10:30:00 GMT

{
  "id": 42,
  "name": "Produit Example",
  "price": 29.99
}

# Requête conditionnelle ultérieure
GET /products/42 HTTP/1.1
If-None-Match: "33a64df551425fcc55e4d42a148795d9f25f89d4"

# Réponse si non modifié
HTTP/1.1 304 Not Modified
```

CONTRAINTE CACHEABLE : OPTIMISATION DES PERFORMANCES (PARTIE 2/3) [A]

Utilisation des headers de cache et requêtes conditionnelles (http) - Suite

```
21. ETag: "33a64df551425fcc55e4d42a148795d9f25f89d4"
```

CONTRAINTE CACHEABLE : OPTIMISATION DES PERFORMANCES (PARTIE 3/3)

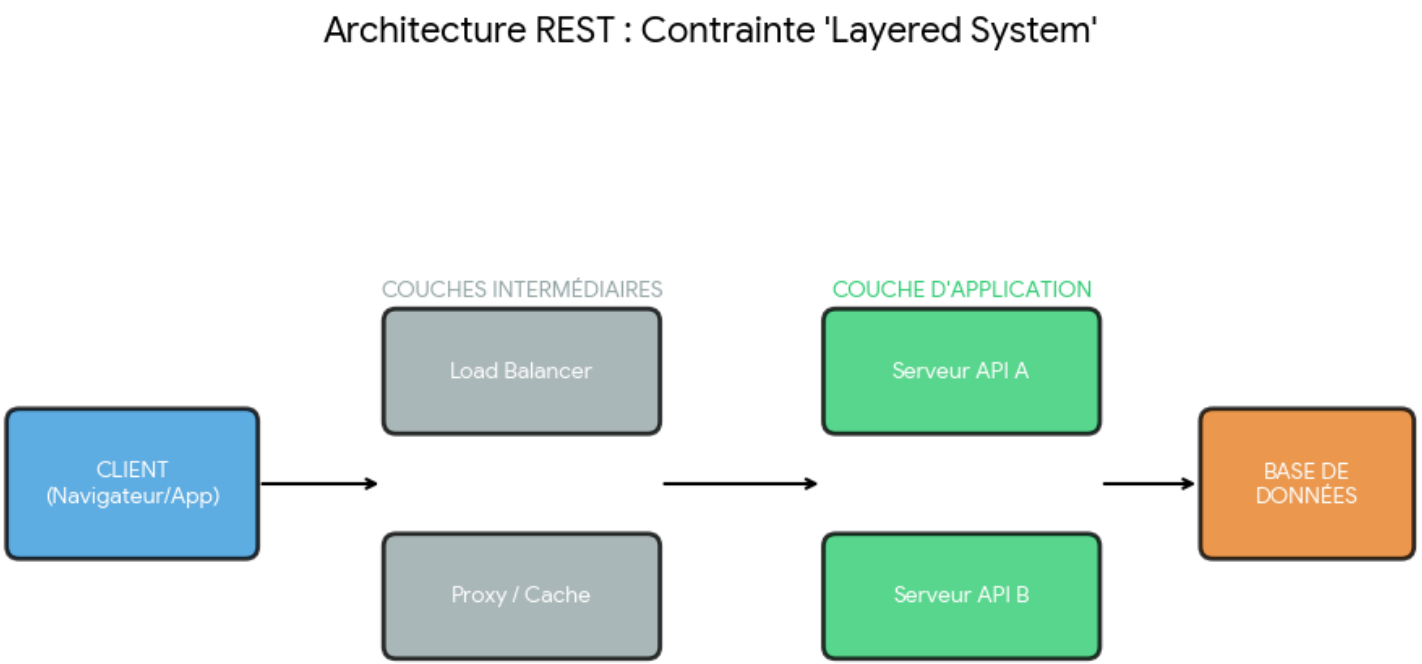
Principales directives Cache-Control

Directive	Type	Description	Cas d'usage
public	Réponse	Peut être mis en cache par n'importe quel cache	Ressources publiques statiques
private	Réponse	Uniquement en cache côté client	Données utilisateur personnalisées
no-cache	Requête/Réponse	Doit revalider avant utilisation	Données sensibles nécessitant validation
no-store	Requête/Réponse	Aucune mise en cache autorisée	Données hautement sensibles
max-age=3600	Réponse	Durée de validité en secondes	Données avec durée de vie connue
must-revalidate	Réponse	Revalidation obligatoire après expiration	Données critiques

CONTRAINTE LAYERED SYSTEM (PARTIE 1/3)

L'architecture en couches (Layered System) permet de composer l'architecture de plusieurs couches hiérarchiques en contraignant le comportement des composants : chaque composant ne peut voir que la couche immédiate avec laquelle il interagit. Cette contrainte permet d'encapsuler les services legacy, d'introduire des intermédiaires (load balancers, caches, proxies) sans que le client ne soit affecté.

CONTRAINTE LAYERED SYSTEM (PARTIE 2/3)



Le client interagit avec une interface unique sans connaître la complexité interne.\nChaque couche ne voit que la couche immédiatement adjacente.

CONTRAINTE LAYERED SYSTEM (PARTIE 3/3)

Avantages de l'architecture en couches :

Encapsulation et isolation des préoccupations

Flexibilité pour ajouter/modifier des couches intermédiaires

Sécurité renforcée (firewall, authentification à différents niveaux)

Scalabilité via load balancing transparent

Mise en cache à plusieurs niveaux

CONTRAINTE LAYERED SYSTEM (PARTIE 3/3)

Composants typiques dans une architecture REST en couches :

CDN (Content Delivery Network) pour les ressources statiques

Reverse Proxy / API Gateway (authentification, rate limiting)

Load Balancer pour distribution de charge

Serveurs d'application (logique métier)

Couche de cache (Redis, Memcached)

Couche de persistance (bases de données)

MÉTHODES HTTP : FONDEMENTS SÉMANTIQUES (PARTIE 1/3)

Les méthodes HTTP (aussi appelées verbes HTTP) définissent l'action à effectuer sur une ressource. REST utilise ces méthodes de manière sémantique : chaque méthode a une signification précise et des propriétés spécifiques (idempotence, sécurité). La compréhension correcte de ces méthodes est essentielle pour concevoir des APIs RESTful cohérentes et prévisibles.

MÉTHODES HTTP : FONDEMENTS SÉMANTIQUES (PARTIE 2/3)

Propriétés des principales méthodes HTTP

Méthode	Sémantique	Idempotent	Safe	Cacheable	Corps requête	Corps réponse
GET	Récupérer une ressource	Oui	Oui	Oui	Non	Oui
POST	Créer une ressource	Non	Non	Conditionnel	Oui	Oui
PUT	Remplacer une ressource	Oui	Non	Non	Oui	Oui
PATCH	Modifier partiellement	Non	Non	Non	Oui	Oui
DELETE	Supprimer une ressource	Oui	Non	Non	Optionnel	Optionnel
HEAD	Récupérer métadonnées	Oui	Oui	Oui	Non	Non
OPTIONS	Obtenir options disponibles	Oui	Oui	Non	Non	Oui

MÉTHODES HTTP : FONDEMENTS SÉMANTIQUES (PARTIE 3/3)

Concepts clés :

Safe (sûre) : La méthode ne modifie pas l'état du serveur (lecture seule)

Idempotent : Plusieurs appels identiques produisent le même résultat qu'un seul appel

Cacheable : La réponse peut être mise en cache pour réutilisation ultérieure

Importance de l'idempotence :

Permet de réessayer automatiquement en cas d'échec réseau

Facilite la gestion des erreurs et la résilience

GET, PUT, DELETE sont idempotents, POST ne l'est pas

GET : RÉCUPÉRATION DE RESSOURCES (PARTIE 1/3)

GET est la méthode la plus courante, utilisée pour récupérer une représentation d'une ressource. Elle est safe (ne modifie pas l'état) et idempotente (plusieurs appels identiques retournent le même résultat). GET ne doit jamais avoir d'effets de bord sur le serveur. Les paramètres sont passés via l'URL (query parameters) et les réponses sont cacheables par défaut.

GET : RÉCUPÉRATION DE RESSOURCES (PARTIE 2/3)

Exemples d'utilisation de GET pour différents scénarios (http)

```
# Récupérer une collection de ressources
GET /api/v1/users HTTP/1.1
Host: api.exemple.com
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json
{
  "users": [
    {"id": 1, "name": "Alice"},
    {"id": 2, "name": "Bob"}
  ],
  "total": 2,
  "page": 1
}

# Récupérer une ressource spécifique
GET /api/v1/users/1 HTTP/1.1

HTTP/1.1 200 OK
```

GET : RÉCUPÉRATION DE RESSOURCES (PARTIE 2/3) [A]

Exemples d'utilisation de GET pour différents scénarios (http) - Suite

```
21. {
22.   "id": 1,
23.   "name": "Alice",
24.   "email": "alice@example.com"
25. }
26.
27. # Filtrage et pagination
28. GET /api/v1/products?category=electronics&page=2&limit=20&sort=price_asc HTTP/1.1
```

GET : RÉCUPÉRATION DE RESSOURCES (PARTIE 3/3)

Codes de statut HTTP appropriés pour GET :

200 OK : Ressource trouvée et retournée avec succès

304 Not Modified : Ressource non modifiée (avec cache conditionnel)

404 Not Found : Ressource demandée n'existe pas

400 Bad Request : Paramètres de requête invalides

401 Unauthorized : Authentification requise

403 Forbidden : Accès refusé (authentifié mais non autorisé)

500 Internal Server Error : Erreur serveur inattendue

POST : CRÉATION DE RESSOURCES (PARTIE 1/3)

POST est utilisée pour créer de nouvelles ressources ou soumettre des données à traiter. Contrairement à PUT, POST n'est ni safe ni idempotent : plusieurs appels identiques créeront plusieurs ressources. Le serveur détermine généralement l'URI de la nouvelle ressource et le retourne dans le header Location. POST peut aussi être utilisé pour des opérations complexes qui ne correspondent pas aux autres méthodes.

POST : CRÉATION DE RESSOURCES (PARTIE 2/3)

Utilisation de POST pour création et opérations complexes (http)

```
# Création d'un nouvel utilisateur
POST /api/v1/users HTTP/1.1
Host: api.exemple.com
Content-Type: application/json

{
  "username": "charlie",
  "email": "charlie@example.com",
  "firstName": "Charlie",
  "lastName": "Brown"
}

# Réponse avec ressource créée
HTTP/1.1 201 Created
Location: /api/v1/users/3
Content-Type: application/json

{
  "id": 3,
  "username": "charlie",
```

POST : CRÉATION DE RESSOURCES (PARTIE 2/3) [A]

Utilisation de POST pour création et opérations complexes (http) - Suite

```
21.   "email": "charlie@example.com",
22.   "firstName": "Charlie",
23.   "lastName": "Brown",
24.   "createdAt": "2024-01-15T14:30:00Z"
25. }
26.
27. # POST pour opération complexe (recherche avancée)
28. POST /api/v1/products/search HTTP/1.1
29. Content-Type: application/json
30.
31. {
32.   "filters": {
33.     "categories": ["electronics", "computers"],
34.     "priceRange": {"min": 100, "max": 1000},
35.     "brands": ["BrandA", "BrandB"]
36.   },
37.   "sort": "price_desc"
38. }
```

POST : CRÉATION DE RESSOURCES (PARTIE 3/3)

Codes de statut HTTP appropriés pour POST :

201 Created : Ressource créée avec succès (inclure header Location)

200 OK : Requête traitée mais pas de nouvelle ressource créée

202 Accepted : Requête acceptée pour traitement asynchrone

400 Bad Request : Données invalides ou manquantes

409 Conflict : Conflit avec l'état actuel (ex: username déjà existant)

422 Unprocessable Entity : Validation métier échouée

413 Payload Too Large : Corps de requête trop volumineux

PUT : REMPLACEMENT COMPLET DE RESSOURCE (PARTIE 1/3)

PUT remplace complètement une ressource existante par la nouvelle représentation fournie. Elle est idempotente : appeler PUT plusieurs fois avec les mêmes données produit le même résultat. PUT nécessite que le client connaisse l'URI complet de la ressource. Si la ressource n'existe pas, PUT peut la créer (selon l'implémentation). Tous les champs de la ressource doivent être fournis, les champs omis seront supprimés ou réinitialisés.

PUT : REMPLACEMENT COMPLET DE RESSOURCE (PARTIE 2/3)

Utilisation de PUT pour remplacement complet (http)

```
1. # Remplacement complet d'un utilisateur
2. PUT /api/v1/users/3 HTTP/1.1
3. Host: api.exemple.com
4. Content-Type: application/json
5.
6. {
7.   "username": "charlie",
8.   "email": "charlie.brown@example.com",
9.   "firstName": "Charles",
10.   "lastName": "Brown",
11.   "isActive": true
12. }
13.
14. # Réponse : ressource mise à jour
15. HTTP/1.1 200 OK
16. Content-Type: application/json
17.
18. {
19.   "id": 3,
20.   "username": "charlie",
```

PUT : REMPLACEMENT COMPLET DE RESSOURCE (PARTIE 2/3) [A]

Utilisation de PUT pour remplacement complet (http) - Suite

```
21.   "email": "charlie.brown@example.com",
22.   "firstName": "Charles",
23.   "lastName": "Brown",
24.   "isActive": true,
25.   "updatedAt": "2024-01-15T15:45:00Z"
26. }
27.
28. # PUT pour créer une ressource (si autorisé)
29. PUT /api/v1/users/999 HTTP/1.1
30. {...}
31.
32. HTTP/1.1 201 Created
33. Location: /api/v1/users/999
```

PUT : REMPLACEMENT COMPLET DE RESSOURCE (PARTIE 3/3)

Codes de statut HTTP appropriés pour

SPÉCIFICATION OPENAPI 3.0 : VUE D'ENSEMBLE (PARTIE 1/2)

OpenAPI 3.0 est un standard de description d'API REST indépendant du langage de programmation. Il permet de définir de manière formelle et exhaustive tous les aspects d'une API : endpoints, paramètres, schémas de données, réponses, authentification et documentation. Le document OpenAPI sert de contrat entre producteurs et consommateurs d'API, facilitant la génération automatique de code, la validation et les tests.

SPÉCIFICATION OPENAPI 3.0 : VUE D'ENSEMBLE (PARTIE 2/2)

Format de fichier : YAML ou JSON

Version actuelle : OpenAPI 3.0.x et 3.1.x

Évolution d'OpenAPI 2.0 (anciennement Swagger)

Contrat machine-readable et human-readable

Écosystème riche d'outils de génération et validation

STRUCTURE DU DOCUMENT OPENAPI 3.0 (PARTIE 1/2)

Structure de base d'un document OpenAPI (yaml)

```yaml
openapi: 3.0.3
info:
  title: API Gestion Utilisateurs
  version: 1.0.0
  description: API REST pour la gestion des utilisateurs
  contact:
    name: Équipe API
    email: api@example.com
servers:
  - url: https://api.example.com/v1
    description: Serveur de production
  - url: https://staging-api.example.com/v1
    description: Serveur de staging
paths:
  # Définition des endpoints
components:
  # Schémas réutilisables
security:
  # Mécanismes d'authentification
```

STRUCTURE DU DOCUMENT OPENAPI 3.0 (PARTIE 2/2)

openapi : Version de la spécification utilisée (obligatoire)

info : Métadonnées de l'API (titre, version, description)

servers : Liste des serveurs disponibles (dev, staging, prod)

paths : Définition de tous les endpoints et opérations

components : Éléments réutilisables (schémas, réponses, paramètres)

security : Schémas de sécurité globaux ou par endpoint

SECTIONS PRINCIPALES DU DOCUMENT OPENAPI

Sections OpenAPI 3.0 et leurs rôles

Section	Obligatoire	Description	Utilisation
openapi	Oui	Version de la spec	Définit la version OpenAPI (3.0.x)
info	Oui	Métadonnées API	Titre, version, description, contact
servers	Non	URLs des serveurs	Environnements multiples (dev, prod)
paths	Oui	Endpoints de l'API	Toutes les routes et opérations HTTP
components	Non	Éléments réutilisables	Schémas, réponses, paramètres
security	Non	Authentification	OAuth2, API Key, JWT
tags	Non	Organisation logique	Groupement des endpoints

DÉFINITION DES ENDPOINTS : SECTION PATHS (PARTIE 1/2)

La section 'paths' constitue le cœur du document OpenAPI. Chaque chemin (path) représente une ressource REST, et pour chaque chemin, on définit les opérations HTTP disponibles (GET, POST, PUT, DELETE, PATCH). Chaque opération spécifie ses paramètres, corps de requête, réponses possibles et codes de statut HTTP.

DÉFINITION DES ENDPOINTS : SECTION PATHS (PARTIE 2/2)

Exemple de définition d'endpoint GET (yaml)

```yaml
paths:
  /users:
    get:
      summary: Récupérer la liste des utilisateurs
      operationId: getUsers
      tags:
        - Users
      parameters:
        - name: page
          in: query
          schema:
            type: integer
            default: 1
        - name: limit
          in: query
          schema:
            type: integer
            default: 20
      responses:
        '200':
```

DÉFINITION DES ENDPOINTS : SECTION PATHS (PARTIE 2/2) [A]

Exemple de définition d'endpoint GET (yaml) - Suite

```
21.           description: Liste des utilisateurs récupérée avec succès
22.           content:
23.             application/json:
24.               schema:
25.                 type: array
26.                 items:
27.                   $ref: '#/components/schemas/User'
28.         '400':
29.           description: Paramètres de requête invalides
```

DÉFINITION DES ENDPOINTS : OPÉRATIONS CRUD

Endpoints avec paramètres de chemin et opérations multiples (yaml)

```yaml
paths:
  /users/{userId}:
    get:
      summary: Récupérer un utilisateur par ID
      parameters:
        - name: userId
          in: path
          required: true
          schema:
            type: integer
      responses:
        '200':
          description: Utilisateur trouvé
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/User'
        '404':
          description: Utilisateur non trouvé
    put:
```

DÉFINITION DES ENDPOINTS : OPÉRATIONS CRUD [A]

Endpoints avec paramètres de chemin et opérations multiples (yaml) - Suite

```
21.       summary: Mettre à jour un utilisateur
22.       parameters:
23.         - name: userId
24.           in: path
25.           required: true
26.           schema:
27.             type: integer
28.       requestBody:
29.         required: true
30.         content:
31.           application/json:
32.             schema:
33.               $ref: '#/components/schemas/UserUpdate'
34.       responses:
35.         '200':
36.           description: Utilisateur mis à jour
37.         '404':
38.           description: Utilisateur non trouvé
39.     delete:
40.       summary: Supprimer un utilisateur
```

DÉFINITION DES ENDPOINTS : OPÉRATIONS CRUD [B]

Endpoints avec paramètres de chemin et opérations multiples (yaml) - Suite

```
41.       parameters:
42.         - name: userId
43.           in: path
44.           required: true
45.           schema:
46.             type: integer
47.       responses:
48.         '204':
49.           description: Utilisateur supprimé
50.         '404':
51.           description: Utilisateur non trouvé
```

SCHÉMAS DE DONNÉES : SECTION COMPONENTS/SCHEMAS (PARTIE 1/2)

Les schémas de données définissent la structure des objets JSON échangés par l'API. Ils sont déclarés dans la section 'components/schemas' pour être réutilisés dans toute la spécification via des références ($ref). Les schémas supportent la validation (types, formats, contraintes) et la documentation des propriétés.

SCHÉMAS DE DONNÉES : SECTION COMPONENTS/SCHEMAS (PARTIE 2/2)

Définition d'un schéma User complet (yaml)

```yaml
components:
  schemas:
    User:
      type: object
      required:
        - id
        - email
        - username
      properties:
        id:
          type: integer
          format: int64
          description: Identifiant unique de l'utilisateur
          example: 12345
        email:
          type: string
          format: email
          description: Adresse email de l'utilisateur
          example: user@example.com
        username:
```

SCHÉMAS DE DONNÉES : SECTION COMPONENTS/SCHEMAS (PARTIE 2/2) [A]

Définition d'un schéma User complet (yaml) - Suite

```
21.           type: string
22.           minLength: 3
23.           maxLength: 50
24.           pattern: '^[a-zA-Z0-9_]+$'
25.           description: Nom d'utilisateur unique
26.           example: john_doe
27.         createdAt:
28.           type: string
29.           format: date-time
30.           description: Date de création du compte
31.           example: '2024-01-15T10:30:00Z'
32.         roles:
33.           type: array
34.           items:
35.             type: string
36.             enum: [admin, user, moderator]
37.           description: Rôles assignés à l'utilisateur
38.           example: [user]
```

TYPES DE DONNÉES ET FORMATS OPENAPI

Types et formats de données OpenAPI 3.0

Type	Format	Description	Exemple
string	-	Chaîne de caractères	"Hello World"
string	date	Date ISO 8601	"2024-01-15"
string	date-time	Date et heure ISO 8601	"2024-01-15T10:30:00Z"
string	email	Adresse email	"user@example.com"
string	uuid	Identifiant UUID	"550e8400-e29b-41d4-a716-446655440000"
integer	int32	Entier 32 bits	42
integer	int64	Entier 64 bits (long)	9223372036854775807
number	float	Nombre à virgule flottante	3.14
number	double	Double précision	3.141592653589793
boolean	-	Booléen	true
array	-	Tableau d'éléments	[1, 2, 3]
object	-	Objet JSON	{"key": "value"}

SCHÉMAS AVANCÉS : COMPOSITION ET HÉRITAGE (PARTIE 1/2)

Composition de schémas avec allOf, oneOf, anyOf (yaml)

```yaml
components:
  schemas:
    # Schéma de base
    BaseEntity:
      type: object
      required:
        - id
        - createdAt
      properties:
        id:
          type: integer
          format: int64
        createdAt:
          type: string
          format: date-time
        updatedAt:
          type: string
          format: date-time

    # Héritage avec allOf
```

SCHÉMAS AVANCÉS : COMPOSITION ET HÉRITAGE (PARTIE 1/2) [A]

Composition de schémas avec allOf, oneOf, anyOf (yaml) - Suite

```
21.     Product:
22.       allOf:
23.         - $ref: '#/components/schemas/BaseEntity'
24.         - type: object
25.           required:
26.             - name
27.             - price
28.           properties:
29.             name:
30.               type: string
31.             price:
32.               type: number
33.               format: double
34.             category:
35.               type: string
36.
37.     # Composition avec oneOf
38.     PaymentMethod:
39.       oneOf:
40.         - $ref: '#/components/schemas/CreditCard'
```

SCHÉMAS AVANCÉS : COMPOSITION ET HÉRITAGE (PARTIE 1/2) [B]

Composition de schémas avec allOf, oneOf, anyOf (yaml) - Suite

```
41.         - $ref: '#/components/schemas/BankTransfer'
42.         - $ref: '#/components/schemas/PayPal'
43.       discriminator:
44.         propertyName: paymentType
```

SCHÉMAS AVANCÉS : COMPOSITION ET HÉRITAGE (PARTIE 2/2)

allOf : Combine plusieurs schémas (héritage, extension)

oneOf : Un seul schéma parmi plusieurs doit correspondre

anyOf : Un ou plusieurs schémas peuvent correspondre

not : Le schéma ne doit pas correspondre

discriminator : Indique la propriété pour distinguer les types

PARAMÈTRES : TYPES ET EMPLACEMENTS (PARTIE 1/2)

OpenAPI distingue quatre emplacements pour les paramètres : path (dans l'URL), query (chaîne de requête), header (en-têtes HTTP), et cookie. Chaque paramètre peut être obligatoire ou optionnel, typé, validé et documenté.

PARAMÈTRES : TYPES ET EMPLACEMENTS (PARTIE 2/2)

Définition complète des paramètres par emplacement (yaml)

```yaml
paths:
  /products/{productId}:
    get:
      summary: Rechercher des produits
      parameters:
        # Paramètre de chemin (obligatoire)
        - name: productId
          in: path
          required: true
          description: Identifiant unique du produit
          schema:
            type: integer
            format: int64

        # Paramètres de requête (optionnels)
        - name: category
          in: query
          required: false
          description: Filtrer par catégorie
          schema:
```

PARAMÈTRES : TYPES ET EMPLACEMENTS (PARTIE 2/2) [A]

Définition complète des paramètres par emplacement (yaml) - Suite

```
21.             type: string
22.             enum: [electronics, clothing, books]
23.
24.         - name: minPrice
25.           in: query
26.           schema:
27.             type: number
28.             format: double
29.             minimum: 0
30.
31.         - name: maxPrice
32.           in: query
33.           schema:
34.             type: number
35.             format: double
36.             minimum: 0
37.
38.         # Paramètre d'en-tête
39.         - name: X-API-Version
40.           in: header
```

PARAMÈTRES : TYPES ET EMPLACEMENTS (PARTIE 2/2) [B]

Définition complète des paramètres par emplacement (yaml) - Suite

```
41.           required: false
42.           schema:
43.             type: string
44.             default: '1.0'
45.
46.         # Paramètre de cookie
47.         - name: sessionId
48.           in: cookie
49.           required: false
50.           schema:
51.             type: string
```

CORPS DE REQUÊTE : REQUESTBODY (PARTIE 1/2)

Définition du corps de requête avec exemples (yaml)

```yaml
paths:
  /users:
    post:
      summary: Créer un nouvel utilisateur
      requestBody:
        description: Données du nouvel utilisateur à créer
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/UserCreate'
            examples:
              exemple1:
                summary: Utilisateur basique
                value:
                  email: newuser@example.com
                  username: newuser123
                  password: SecurePass123!
              exemple2:
                summary: Utilisateur avec rôles
```

CORPS DE REQUÊTE : REQUESTBODY (PARTIE 1/2) [A]

Définition du corps de requête avec exemples (yaml) - Suite

```
21.                 value:
22.                   email: admin@example.com
23.                   username: admin_user
24.                   password: AdminPass456!
25.                   roles: [admin, moderator]
26.           application/xml:
27.             schema:
28.               $ref: '#/components/schemas/UserCreate'
29.       responses:
30.         '201':
31.           description: Utilisateur créé avec succès
32.           content:
33.             application/json:
34.               schema:
35.                 $ref: '#/components/schemas/User'
36.         '400':
37.           description: Données invalides
```

CORPS DE REQUÊTE : REQUESTBODY (PARTIE 2/2)

required : Indique si le corps est obligatoire

content : Types MIME supportés (application/json, application/xml, etc.)

schema : Structure des données attendues

examples : Exemples multiples pour documentation et tests

Multipart/form-data : Pour upload de fichiers

DÉFINITION DES RÉPONSES HTTP

Définition complète des réponses avec en-têtes (yaml)

```yaml
paths:
  /users/{userId}:
    get:
      responses:
        '200':
          description: Utilisateur récupéré avec succès
          headers:
            X-RateLimit-Limit:
              schema:
                type: integer
              description: Nombre de requêtes autorisées par heure
            X-RateLimit-Remaining:
              schema:
                type: integer
              description: Nombre de requêtes restantes
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/User'
        '404':
```

DÉFINITION DES RÉPONSES HTTP [A]

Définition complète des réponses avec en-têtes (yaml) - Suite

```
21.           description: Utilisateur non trouvé
22.           content:
23.             application/json:
24.               schema:
25.                 $ref: '#/components/schemas/Error'
26.         '500':
27.           description: Erreur serveur interne
28.           content:
29.             application/json:
30.               schema:
31.                 $ref: '#/components/schemas/Error'
32.         default:
33.           description: Erreur inattendue
34.           content:
35.             application/json:
36.               schema:
37.                 $ref: '#/components/schemas/Error'
38.
39. components:
40.   schemas:
```

DÉFINITION DES RÉPONSES HTTP [B]

Définition complète des réponses avec en-têtes (yaml) - Suite

```
41.     Error:
42.       type: object
43.       required:
44.         - code
45.         - message
46.       properties:
47.         code:
48.           type: string
49.         message:
50.           type: string
51.         details:
52.           type: array
53.           items:
54.             type: string
```

RÉPONSES RÉUTILISABLES : COMPONENTS/RESPONSES (PARTIE 1/2)

Réutilisation de réponses communes (yaml)

```yaml
components:
  responses:
    NotFound:
      description: La ressource demandée n'a pas été trouvée
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/Error'
          example:
            code: NOT_FOUND
            message: La ressource demandée n'existe pas

    Unauthorized:
      description: Authentification requise
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/Error'
          example:
            code: UNAUTHORIZED
```

RÉPONSES RÉUTILISABLES : COMPONENTS/RESPONSES (PARTIE 1/2) [A]

Réutilisation de réponses communes (yaml) - Suite

```
21.             message: Token d'authentification manquant ou invalide
22.
23.     ValidationError:
24.       description: Erreur de validation des données
25.       content:
26.         application/json:
27.           schema:
28.             $ref: '#/components/schemas/ValidationError'
29.
30. paths:
31.   /users/{userId}:
32.     get:
33.       responses:
34.         '200':
35.           description: Succès
36.           content:
37.             application/json:
38.               schema:
39.                 $ref: '#/components/schemas/User'
40.         '404':
```

RÉPONSES RÉUTILISABLES : COMPONENTS/RESPONSES (PARTIE 1/2) [B]

Réutilisation de réponses communes (yaml) - Suite

```
41.           $ref: '#/components/responses/NotFound'
42.         '401':
43.           $ref: '#/components/responses/Unauthorized'
```

RÉPONSES RÉUTILISABLES : COMPONENTS/RESPONSES (PARTIE 2/2)

Évite la duplication des définitions de réponses

Assure la cohérence des messages d'erreur

Facilite la maintenance et les modifications

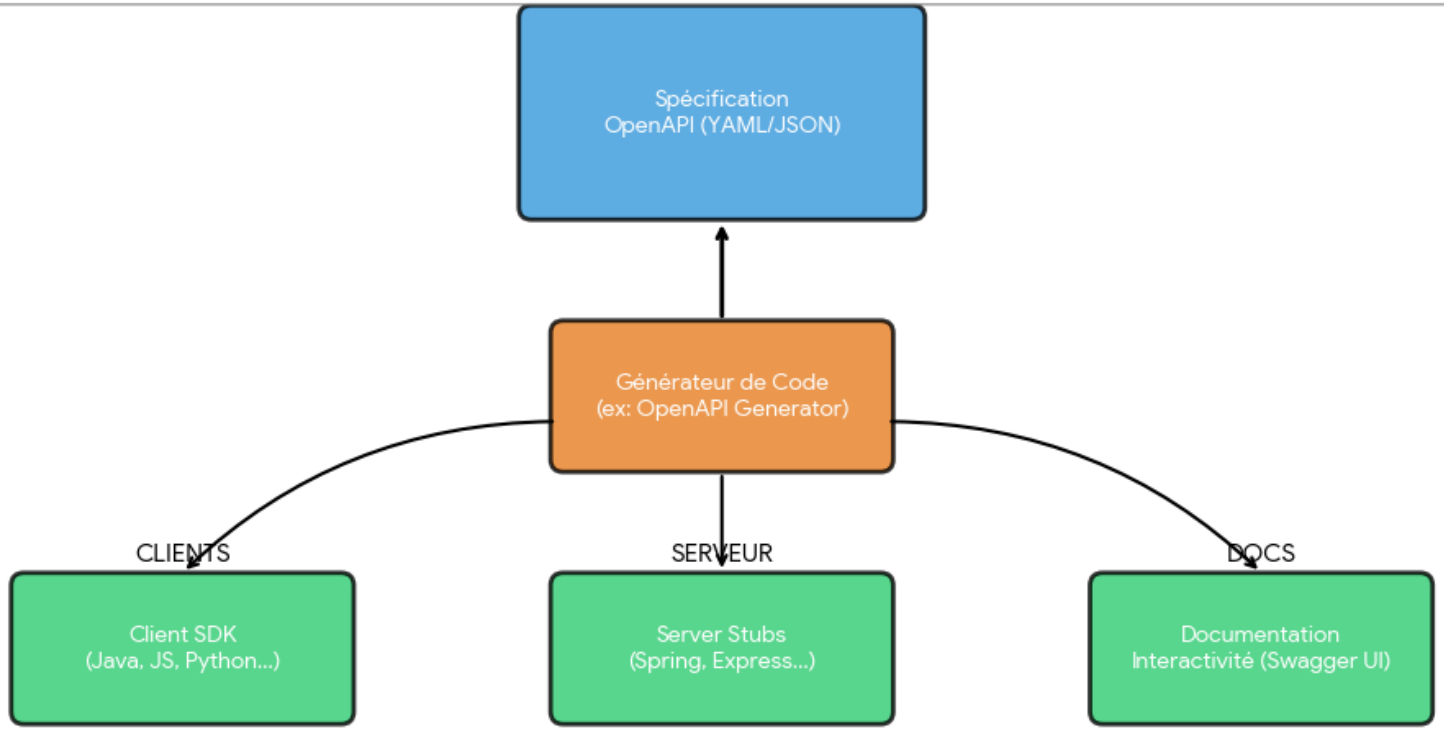
Permet de standardiser les codes d'erreur

Améliore la lisibilité de la spécification

IMAGE : ARCHITECTURE OPENAPI ET GÉNÉRATION DE CODE

Architecture OpenAPI et Workflow de Génération de Code

À partir d'un fichier de définition unique, on génère automatiquement le code client, la structure du serveur et une documentation toujours synchronisée.



GÉNÉRATION DE CODE : VUE D'ENSEMBLE (PARTIE 1/2)

La génération automatique de code à partir de spécifications OpenAPI est un pilier de l'approche API-First. Elle permet de produire automatiquement des stubs de serveur (squelettes de code backend), des clients SDK, de la documentation interactive et des tests, garantissant ainsi que l'implémentation reste fidèle au contrat d'interface défini.

GÉNÉRATION DE CODE : VUE D'ENSEMBLE (PARTIE 2/2)

Stubs backend : Code serveur avec routes, validation, sérialisation

Clients SDK : Bibliothèques pour consommer l'API dans différents langages

Documentation : Interface interactive (Swagger UI, ReDoc)

Tests : Suites de tests automatisés basés sur les exemples

Modèles de données : Classes/structures correspondant aux schémas

Validation : Code de validation automatique des requêtes/réponses

OUTILS DE GÉNÉRATION DE CODE

Principaux outils de génération OpenAPI

Outil	Type	Langages supportés	Caractéristiques
OpenAPI Generator	CLI/Maven/Gradle	50+ langages	Fork de Swagger Codegen, très actif, personnalisable
Swagger Codegen	CLI/Maven/Gradle	40+ langages	Outil original, moins maintenu
NSwag	CLI/.NET	C#, TypeScript	Spécialisé .NET, génération client/serveur
OpenAPI Tools	NPM/CLI	TypeScript, JavaScript	Outils modernes pour écosystème Node.js
Kiota	CLI	C#, Java, TypeScript, Python	Microsoft, focus sur clients SDK
oapi-codegen	CLI/Go	Go	Génération native Go, performant
FastAPI	Framework Python	Python	Génération automatique de spec depuis code

OPENAPI GENERATOR : INSTALLATION ET USAGE

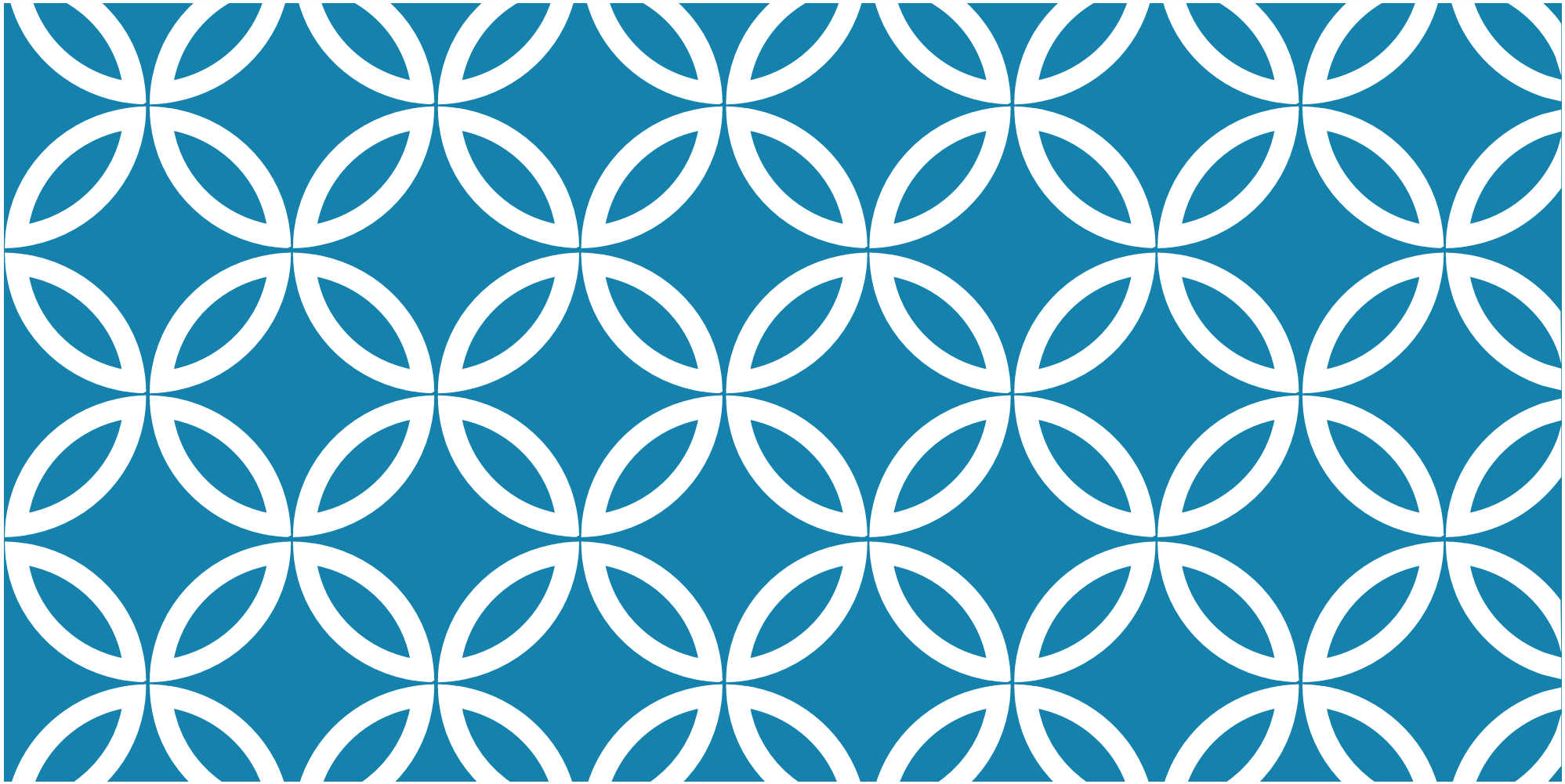
Installation et commandes OpenAPI Generator (bash)

```bash
# Installation via NPM
npm install @openapitools/openapi-generator-cli -g

# Installation via Homebrew (macOS)
brew install openapi-generator

# Installation via Docker
docker pull openapitools/openapi-generator-cli

# Commande de base pour générer du code
openapi-generator-cli generate \
  -i openapi.yaml \
  -g spring \
  -o ./generated-server

# Lister tous les générateurs disponibles
openapi-generator-cli list

# Obtenir les options d'un générateur spécifique
openapi-generator-cli config-help -g spring
```

OPENAPI GENERATOR : INSTALLATION ET USAGE [A]

Installation et commandes OpenAPI Generator (bash) - Suite

```
21.
22. # Génération avec configuration personnalisée
23. openapi-generator-cli generate \
24.   -i openapi.yaml \
25.   -g spring \
26.   -o ./generated-server \
27.   --additional-properties=basePackage=com.example.api,
configPackage=com.example.api.config
```



LECTURE 3 : IMPLÉMENTATION, PERSISTANCE ET DOCUMENTATION D'API REST

- Manipulation avancée du format JSON : sérialisation, désérialisation, validation de schémas et gestion des types de données complexes, - Gestion de la persistance des données : intégration avec base

MANIPULATION AVANCÉE DU FORMAT JSON (PARTIE 1/2)

Le JSON (JavaScript Object Notation) est le format d'échange de données standard pour les API REST. La manipulation efficace du JSON implique la maîtrise de la sérialisation (conversion d'objets en JSON), de la désérialisation (conversion de JSON en objets), et de la validation pour garantir l'intégrité des données.

MANIPULATION AVANCÉE DU FORMAT JSON (PARTIE 2/2)

Sérialisation : transformation d'objets métier en chaînes JSON pour transmission HTTP

Désérialisation : parsing du JSON reçu et reconstruction d'objets typés

Validation de schémas : vérification de la conformité des données selon des règles définies

Gestion des types complexes : dates, énumérations, collections imbriquées, références circulaires

SÉRIALISATION ET DÉSÉRIALISATION EN JAVA (JACKSON) (PARTIE 1/2)

Sérialisation/Désérialisation avec Jackson (java)

```
1. // Sérialisation d'un objet en JSON
2. public class User {
3.     private Long id;
4.     private String username;
5.     @JsonFormat(pattern = "yyyy-MM-dd'T'HH:mm:ss")
6.     private LocalDateTime createdAt;
7.     @JsonIgnore
8.     private String password;
9.     // getters/setters...
10. }
11.
12. ObjectMapper mapper = new ObjectMapper();
13. mapper.registerModule(new JavaTimeModule());
14.
15. User user = new User(1L, "john.doe", LocalDateTime.now());
16. String jsonString = mapper.writeValueAsString(user);
17.
18. // Désérialisation de JSON en objet
19. String json = "{\"id\":1,\"username\":\"john.doe\"}";
20. User deserializedUser = mapper.readValue(json, User.class);
```

SÉRIALISATION ET DÉSÉRIALISATION EN JAVA (JACKSON) (PARTIE 2/2)

Annotations Jackson courantes :

- @JsonProperty : personnaliser le nom du champ JSON
- @JsonIgnore : exclure un champ de la sérialisation
- @JsonFormat : formater dates et nombres
- @JsonInclude : contrôler l'inclusion des valeurs null
- @JsonSerialize/@JsonDeserialize : sérialiseurs personnalisés

VALIDATION DE SCHÉMAS JSON (PARTIE 1/2)

La validation de schémas JSON garantit que les données reçues respectent la structure et les contraintes attendues avant traitement. Cela prévient les erreurs, améliore la sécurité et fournit des messages d'erreur explicites aux clients de l'API.

VALIDATION DE SCHÉMAS JSON (PARTIE 2/2)

Validation avec Bean Validation (java)

```
 1. // Validation avec Bean Validation (JSR-380)
 2. import javax.validation.constraints.*;
 3.
 4. public class CreateUserRequest {
 5.     @NotBlank(message = "Le nom d'utilisateur est obligatoire")
 6.     @Size(min = 3, max = 50)
 7.     private String username;
 8.
 9.     @Email(message = "Format d'email invalide")
10.     @NotNull
11.     private String email;
12.
13.     @Pattern(regexp = "^(?=.*[A-Z])(?=.*[0-9]).{8,}$",
14.              message = "Le mot de passe doit contenir 8 caractères,
une majuscule et un chiffre")
15.     private String password;
16.
17.     @Min(18)
18.     @Max(120)
19.     private Integer age;
```

VALIDATION DE SCHÉMAS JSON (PARTIE 2/2) [A]

Validation avec Bean Validation (java) - Suite

```
21.     @Valid // Validation en cascade
22.     private Address address;
23. }
24.
25. // Dans le contrôleur
26. @PostMapping("/users")
27. public ResponseEntity<?> createUser(@Valid @RequestBody CreateUserRequest
request) {
28.     // Si validation échoue, Spring retourne automatiquement 400 Bad Request
29.     return ResponseEntity.ok(userService.create(request));
30. }
```

GESTION DES TYPES DE DONNÉES COMPLEXES (PARTIE 1/2)

Stratégies de Gestion des Types Complexes en JSON

Type de Données	Problématique	Solution Recommandée
Dates/Timestamps	Formats multiples, fuseaux horaires	ISO 8601 (yyyy-MM-dd'T'HH:mm:ssZ), UTC
Énumérations	Lisibilité vs robustesse	Chaînes de caractères avec validation
BigDecimal/Money	Précision des décimales	Chaînes avec parsing côté serveur
Collections imbriquées	Profondeur et performance	Limiter la profondeur, pagination
Références circulaires	Boucles infinies	@JsonIdentityInfo ou DTOs dédiés
Fichiers binaires	Taille et encodage	Base64 ou URLs de ressources séparées

GESTION DES TYPES DE DONNÉES COMPLEXES (PARTIE 2/2)

Gestion des Références Circulaires (java)

```
 1. // Gestion des références circulaires
 2. @Entity
 3. @JsonIdentityInfo(generator = ObjectIdGenerators.PropertyGenerator.class
, property = "id")
 4. public class Department {
 5.     @Id
 6.     private Long id;
 7.     private String name;
 8.
 9.     @OneToMany(mappedBy = "department")
10.     private List<Employee> employees;
11. }
12.
13. @Entity
14. @JsonIdentityInfo(generator = ObjectIdGenerators.PropertyGenerator.class
, property = "id")
15. public class Employee {
16.     @Id
17.     private Long id;
18.     private String name;
19.
20.     @ManyToOne
```

GESTION DES TYPES DE DONNÉES COMPLEXES (PARTIE 2/2) [A]

Gestion des Références Circulaires (java) - Suite

```
21.     private Department department;
22. }
23.
24. // Alternative : utiliser des DTOs pour contrôler la sérialisation
25. public class DepartmentDTO {
26.     private Long id;
27.     private String name;
28.     private List<Long> employeeIds; // Seulement les IDs
29. }
```

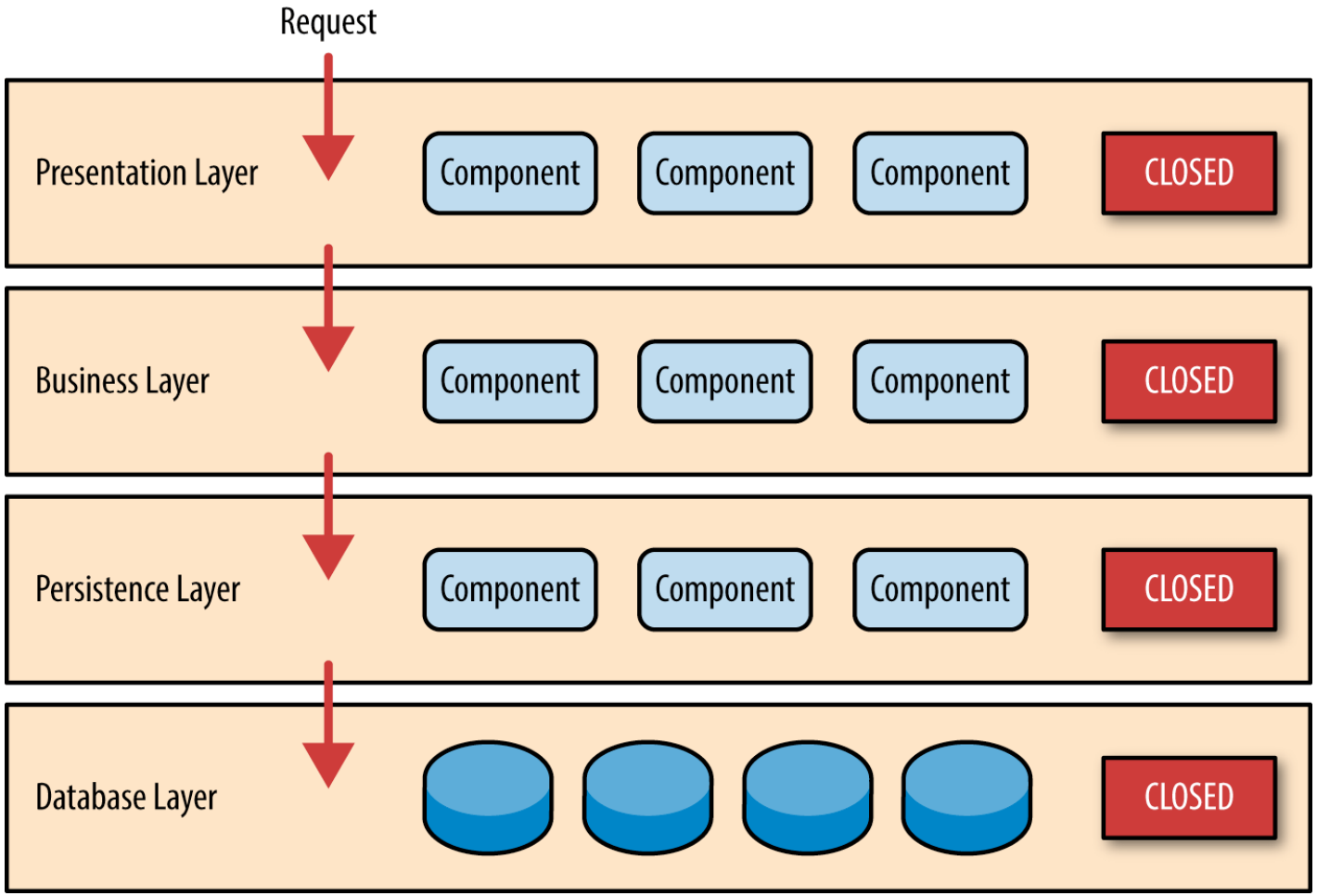
GESTION DE LA PERSISTANCE : ARCHITECTURE (PARTIE 1/4)

Intégration avec Bases de Données

GESTION DE LA PERSISTANCE : ARCHITECTURE (PARTIE 2/4)

La persistance des données dans une API REST nécessite une architecture en couches bien définie pour séparer les responsabilités. L'approche standard utilise le pattern Repository pour abstraire l'accès aux données et un ORM pour mapper les objets métier aux tables relationnelles.

GESTION DE LA PERSISTANCE : ARCHITECTURE (PARTIE 3/4)



GESTION DE LA PERSISTANCE : ARCHITECTURE (PARTIE 4/4)

Avantages de l'architecture en couches :

- Séparation des préoccupations (SoC)
- Testabilité accrue (mocking facile)
- Réutilisabilité du code métier
- Facilité de maintenance et d'évolution
- Indépendance vis-à-vis de la technologie de persistance

ORM ET JPA : CONCEPTS FONDAMENTAUX (PARTIE 1/2)

L'ORM (Object-Relational Mapping) permet de mapper automatiquement les classes Java aux tables de base de données, éliminant le besoin d'écrire du SQL manuel. JPA (Java Persistence API) est la spécification standard, avec Hibernate comme implémentation la plus populaire.

ORM ET JPA : CONCEPTS FONDAMENTAUX (PARTIE 2/2)

Entité JPA avec Annotations (java)

```
 1. // Définition d'une entité JPA
 2. import javax.persistence.*;
 3. import java.time.LocalDateTime;
 4.
 5. @Entity
 6. @Table(name = "users", indexes = {
 7.     @Index(name = "idx_email", columnList = "email", unique = true)
 8. })
 9. public class User {
10.     @Id
11.     @GeneratedValue(strategy = GenerationType.IDENTITY)
12.     private Long id;
13.
14.     @Column(nullable = false, length = 100)
15.     private String username;
16.
17.     @Column(nullable = false, unique = true)
18.     private String email;
19.
20.     @Column(name = "password_hash", nullable = false)
```

ORM ET JPA : CONCEPTS FONDAMENTAUX (PARTIE 2/2) [A]

Entité JPA avec Annotations (java) - Suite

```
21.     private String passwordHash;
22.
23.     @Enumerated(EnumType.STRING)
24.     @Column(length = 20)
25.     private UserRole role;
26.
27.     @Column(name = "created_at", nullable = false, updatable = false)
28.     private LocalDateTime createdAt;
29.
30.     @Column(name = "updated_at")
31.     private LocalDateTime updatedAt;
32.
33.     @PrePersist
34.     protected void onCreate() {
35.         createdAt = LocalDateTime.now();
36.     }
37.
38.     @PreUpdate
39.     protected void onUpdate() {
40.         updatedAt = LocalDateTime.now();
```

ORM ET JPA : CONCEPTS FONDAMENTAUX (PARTIE 2/2) [B]

Entité JPA avec Annotations (java) - Suite

```
41.     }
42.
43.     // Constructeurs, getters, setters...
44. }
```

RELATIONS JPA : ONE-TO-MANY ET MANY-TO-ONE (PARTIE 1/2)

Relations JPA Bidirectionnelles (java)

```
1. // Relation bidirectionnelle One-to-Many / Many-to-One
2. @Entity
3. public class Blog {
4.     @Id
5.     @GeneratedValue(strategy = GenerationType.IDENTITY)
6.     private Long id;
7.
8.     private String title;
9.
10.     @OneToMany(mappedBy = "blog", cascade = CascadeType.ALL, orphanRemoval = true
11.     @JsonManagedReference // Évite les références circulaires
12.     private List<Post> posts = new ArrayList<>();
13.
14.     // Méthodes helper pour maintenir la cohérence bidirectionnelle
15.     public void addPost(Post post) {
16.         posts.add(post);
17.         post.setBlog(this);
18.     }
19.
20.     public void removePost(Post post) {
```

RELATIONS JPA : ONE-TO-MANY ET MANY-TO-ONE (PARTIE 1/2) [A]

Relations JPA Bidirectionnelles (java) - Suite

```
21.         posts.remove(post);
22.         post.setBlog(null);
23.     }
24. }
25.
26. @Entity
27. public class Post {
28.     @Id
29.     @GeneratedValue(strategy = GenerationType.IDENTITY)
30.     private Long id;
31.
32.     private String content;
33.
34.     @ManyToOne(fetch = FetchType.LAZY)
35.     @JoinColumn(name = "blog_id", nullable = false)
36.     @JsonBackReference // Évite les références circulaires
37.     private Blog blog;
38. }
```

RELATIONS JPA : ONE-TO-MANY ET MANY-TO-ONE (PARTIE 2/2)

Bonnes pratiques pour les relations JPA :

- Utiliser FetchType.LAZY par défaut pour éviter le chargement excessif
- Définir cascade approprié (ALL, PERSIST, MERGE, REMOVE)
- Utiliser orphanRemoval = true pour supprimer automatiquement les entités orphelines
- Maintenir la cohérence bidirectionnelle avec des méthodes helper
- Éviter les N+1 queries avec @EntityGraph ou fetch joins

PATTERN REPOSITORY AVEC SPRING DATA JPA (PARTIE 1/2)

Le pattern Repository abstrait la logique d'accès aux données et fournit une interface orientée collection pour manipuler les entités. Spring Data JPA génère automatiquement l'implémentation des méthodes de base et permet de définir des requêtes personnalisées de manière déclarative.

PATTERN REPOSITORY AVEC SPRING DATA JPA (PARTIE 2/2)

Repository Spring Data JPA (java)

```
 1. import org.springframework.data.jpa.repository.*;
 2. import org.springframework.data.repository.query.Param;
 3. import java.util.List;
 4. import java.util.Optional;
 5.
 6. public interface UserRepository extends JpaRepository<User, Long> {
 7.
 8.     // Méthode de requête dérivée (query derivation)
 9.     Optional<User> findByEmail(String email);
10.
11.     List<User> findByRoleAndCreatedAtAfter(UserRole role, LocalDateTime date);
12.
13.     boolean existsByUsername(String username);
14.
15.     // Requête JPQL personnalisée
16.     @Query("SELECT u FROM User u WHERE u.username LIKE %:keyword%
 OR u.email LIKE %:keyword%")
17.     List<User> searchByKeyword(@Param("keyword") String keyword);
18.
19.     // Requête native SQL
20.     @Query(value = "SELECT * FROM users WHERE role = ?1
ORDER BY created_at DESC LIMIT ?2",
```

PATTERN REPOSITORY AVEC SPRING DATA JPA (PARTIE 2/2) [A]

Repository Spring Data JPA (java) - Suite

```
21.             nativeQuery = true)
22.     List<User> findTopUsersByRole(String role, int limit);
23.
24.     // Requête avec projection
25.     @Query("SELECT new com.example.dto.UserSummaryDTO(u.id, u.username,
 u.email) FROM User u")
26.     List<UserSummaryDTO> findAllSummaries();
27.
28.     // Modification avec @Modifying
29.     @Modifying
30.     @Query("UPDATE User u SET u.lastLoginAt = :loginTime WHERE u.id = :userId")
31.     void updateLastLogin(@Param("userId") Long userId,
 @Param("loginTime") LocalDateTime loginTime);
32. }
```

IMPLÉMENTATION COMPLÈTE : SERVICE LAYER

Service Layer avec Gestion Transactionnelle (java)

```
 1. @Service
 2. @Transactional
 3. public class UserService {
 4.
 5.     private final UserRepository userRepository;
 6.     private final PasswordEncoder passwordEncoder;
 7.
 8.     public UserService(UserRepository userRepository, PasswordEncoder
 passwordEncoder) {
 9.         this.userRepository = userRepository;
10.         this.passwordEncoder = passwordEncoder;
11.     }
12.
13.     @Transactional(readOnly = true)
14.     public List<UserDTO> getAllUsers() {
15.         return userRepository.findAll().stream()
16.             .map(this::convertToDTO)
17.             .collect(Collectors.toList());
18.     }
19.
20.     @Transactional(readOnly = true)
```

IMPLÉMENTATION COMPLÈTE : SERVICE LAYER [A]

Service Layer avec Gestion Transactionnelle (java) - Suite

```
21.     public UserDTO getUserById(Long id) {
22.         User user = userRepository.findById(id)
23.             .orElseThrow(() -> new ResourceNotFoundException("User not
 found with id: " + id));
24.         return convertToDTO(user);
25.     }
26.
27.     public UserDTO createUser(CreateUserRequest request) {
28.         // Validation métier
29.         if (userRepository.existsByUsername(request.getUsername())) {
30.             throw new BusinessException("Username already exists");
31.         }
32.
33.         User user = new User();
34.         user.setUsername(request.getUsername());
35.         user.setEmail(request.getEmail());
36.         user.setPasswordHash(passwordEncoder.encode(request.getPassword()));
37.         user.setRole(UserRole.USER);
38.
39.         User savedUser = userRepository.save(user);
40.         return convertToDTO(savedUser);
```

IMPLÉMENTATION COMPLÈTE : SERVICE LAYER [B]

Service Layer avec Gestion Transactionnelle (java) - Suite

```
41.     }
42.
43.     public UserDTO updateUser(Long id, UpdateUserRequest request) {
44.         User user = userRepository.findById(id)
45.             .orElseThrow(() -> new ResourceNotFoundException("User not found"));
46.
47.         user.setUsername(request.getUsername());
48.         user.setEmail(request.getEmail());
49.
50.         User updatedUser = userRepository.save(user);
51.         return convertToDTO(updatedUser);
52.     }
53.
54.     public void deleteUser(Long id) {
55.         if (!userRepository.existsById(id)) {
56.             throw new ResourceNotFoundException("User not found");
57.         }
58.         userRepository.deleteById(id);
59.     }
60.
```

IMPLÉMENTATION COMPLÈTE : SERVICE LAYER [C]

Service Layer avec Gestion Transactionnelle (java) - Suite

```
61.     private UserDTO convertToDTO(User user) {
62.         return new UserDTO(user.getId(), user.getUsername(),
user.getEmail(), user.getRole());
63.     }
64. }
```

SÉCURISATION DES API REST : VUE D'ENSEMBLE (PARTIE 1/4)

Sécurité des API REST

SÉCURISATION DES API REST : VUE D'ENSEMBLE (PARTIE 2/4)

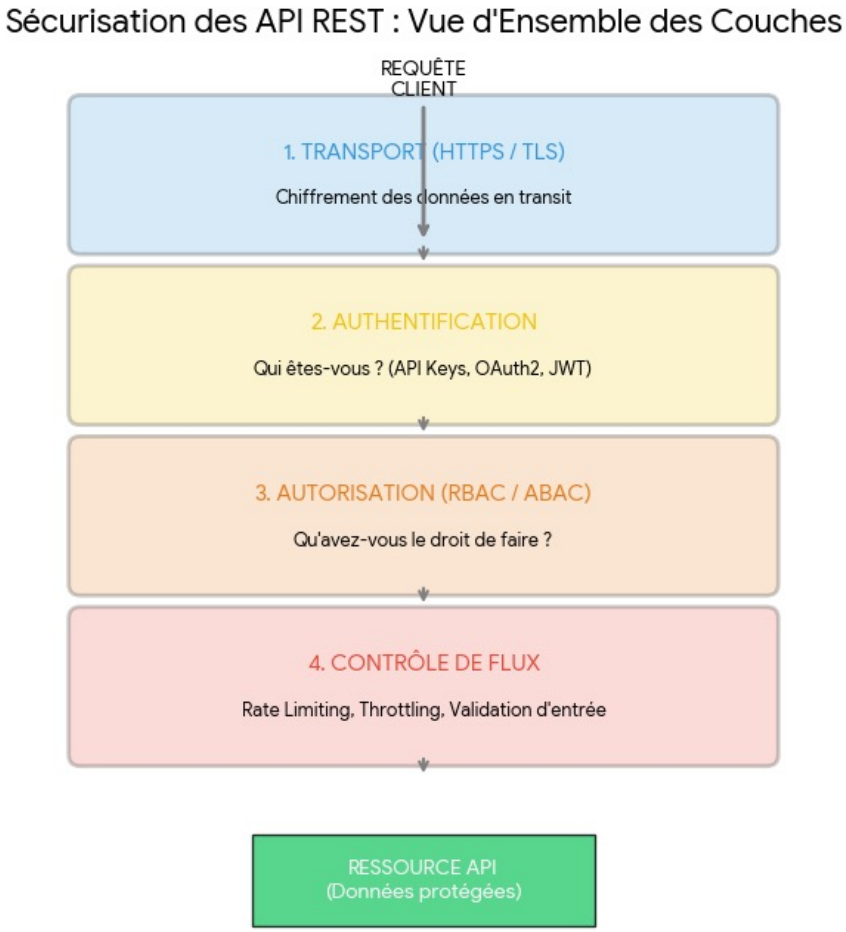
La sécurisation d'une API REST implique trois piliers fondamentaux : l'authentification (vérifier l'identité), l'autorisation (contrôler les permissions), et la protection contre les vulnérabilités. Une API non sécurisée expose l'application à des risques majeurs : vol de données, injection, déni de service, et compromission complète du système.

SÉCURISATION DES API REST : VUE D'ENSEMBLE (PARTIE 3/4)

Principes de sécurité pour API REST :

- Toujours utiliser HTTPS/TLS pour chiffrer les communications
- Implémenter une authentification forte (multi-facteurs si possible)
- Appliquer le principe du moindre privilège pour les autorisations
- Valider et assainir toutes les entrées utilisateur
- Limiter le taux de requêtes (rate limiting) contre les abus
- Logger les accès et surveiller les comportements suspects
- Maintenir les dépendances à jour pour corriger les failles

SÉCURISATION DES API REST : VUE D'ENSEMBLE (PARTIE 4/4)



AUTHENTIFICATION AVEC JWT (JSON WEB TOKEN) (PARTIE 1/3)

JWT est un standard ouvert (RFC 7519) pour créer des tokens d'accès sécurisés et auto-contenus. Contrairement aux sessions serveur, JWT est stateless : toutes les informations nécessaires sont dans le token, permettant une scalabilité horizontale. Le token contient trois parties encodées en Base64 : header (algorithme), payload (claims), et signature (vérification d'intégrité).

AUTHENTIFICATION AVEC JWT (JSON WEB TOKEN) (PARTIE 2/3)

Comparaison Sessions vs JWT

Critère	Sessions Serveur	JWT
Stockage état	Serveur (mémoire/Redis)	Client (token auto-contenu)
Scalabilité	Difficile (sticky sessions)	Excellente (stateless)
Révocation	Immédiate	Difficile (blacklist nécessaire)
Taille requête	Cookie léger	Token plus volumineux
Sécurité	Vulnérable CSRF	Vulnérable XSS si mal stocké
Cas d'usage	Applications monolithiques	Microservices, SPA, Mobile

AUTHENTIFICATION AVEC JWT (JSON WEB TOKEN) (PARTIE 3/3)

Génération et Validation de JWT (java)

```
 1. import io.jsonwebtoken.*;
 2. import io.jsonwebtoken.security.Keys;
 3. import java.security.Key;
 4. import java.util.Date;
 5.
 6. @Service
 7. public class JwtTokenProvider {
 8.
 9.     private final Key secretKey = Keys.secretKeyFor(SignatureAlgorithm.HS512);
10.     private final long validityInMilliseconds = 3600000; // 1 heure
11.
12.     public String createToken(String username, List<String> roles) {
13.         Claims claims = Jwts.claims().setSubject(username);
14.         claims.put("roles", roles);
15.
16.         Date now = new Date();
17.         Date validity = new Date(now.getTime() + validityInMilliseconds);
18.
19.         return Jwts.builder()
20.             .setClaims(claims)
```

AUTHENTIFICATION AVEC JWT (JSON WEB TOKEN) (PARTIE 3/3) [A]

Génération et Validation de JWT (java) - Suite

```
21.             .setIssuedAt(now)
22.             .setExpiration(validity)
23.             .signWith(secretKey, SignatureAlgorithm.HS512)
24.             .compact();
25.     }
26.
27.     public boolean validateToken(String token) {
28.         try {
29.             Jwts.parserBuilder()
30.                 .setSigningKey(secretKey)
31.                 .build()
32.                 .parseClaimsJws(token);
33.             return true;
34.         } catch (JwtException | IllegalArgumentException e) {
35.             return false;
36.         }
37.     }
38.
39.     public String getUsername(String token) {
40.         return Jwts.parserBuilder()
```

AUTHENTIFICATION AVEC JWT (JSON WEB TOKEN) (PARTIE 3/3) [B]

Génération et Validation de JWT (java) - Suite

```
41.             .setSigningKey(secretKey)
42.             .build()
43.             .parseClaimsJws(token)
44.             .getBody()
45.             .getSubject();
46.     }
47.
48.     public List<String> getRoles(String token) {
49.         return (List<String>) Jwts.parserBuilder()
50.             .setSigningKey(secretKey)
51.             .build()
52.             .parseClaimsJws(token)
53.             .getBody()
54.             .get("roles");
55.     }
56. }
```

IMPLÉMENTATION DE L'AUTHENTIFICATION JWT AVEC SPRING SECURITY

Filtre JWT Spring Security (java)

```
1. // Filtre JWT pour intercepter les requêtes
2. public class JwtAuthenticationFilter extends OncePerRequestFilter {
3.
4.     private final JwtTokenProvider tokenProvider;
5.
6.     @Override
7.     protected void doFilterInternal(HttpServletRequest request,
8.                                     HttpServletResponse response,
9.                                     FilterChain filterChain) throws
ServletException, IOException {
10.         try {
11.             String token = extractTokenFromRequest(request);
12.
13.             if (token != null && tokenProvider.validateToken(token)) {
14.                 String username = tokenProvider.getUsername(token);
15.                 List<String> roles = tokenProvider.getRoles(token);
16.
17.                 List<GrantedAuthority> authorities = roles.stream()
18.                     .map(SimpleGrantedAuthority::new)
19.                     .collect(Collectors.toList());
```

IMPLÉMENTATION DE L'AUTHENTIFICATION JWT AVEC SPRING SECURITY [A]

Filtre JWT Spring Security (java) - Suite

```
21.                 UsernamePasswordAuthenticationToken authentication =
22.                     new UsernamePasswordAuthenticationToken(username,
null, authorities);
23.
24.                 SecurityContextHolder.getContext()
.setAuthentication(authentication);
25.             }
26.         } catch (Exception e) {
27.             logger.error("Cannot set user authentication", e);
28.         }
29.
30.         filterChain.doFilter(request, response);
31.     }
32.
33.     private String extractTokenFromRequest(HttpServletRequest request) {
34.         String bearerToken = request.getHeader("Authorization");
35.         if (bearerToken != null && bearerToken.startsWith("Bearer ")) {
36.             return bearerToken.substring(7);
37.         }
38.         return null;
39.     }
40. }
```

DOCUMENTATION INTERACTIVE AVEC SWAGGER UI (PARTIE 1/2)

Swagger UI est un outil puissant qui génère automatiquement une interface web interactive à partir d'une spécification OpenAPI. Il permet aux développeurs et aux consommateurs d'API de visualiser, tester et interagir avec les endpoints sans écrire de code client. Cette documentation vivante s'actualise automatiquement avec les modifications de la spécification, garantissant une documentation toujours à jour.

DOCUMENTATION INTERACTIVE AVEC SWAGGER UI (PARTIE 2/2)

Génération automatique de documentation interactive à partir de la spécification OpenAPI

Interface de test intégrée permettant d'exécuter des requêtes directement depuis le navigateur

Visualisation claire des schémas de données, paramètres et réponses

Support de l'authentification (API Keys, OAuth2, JWT)

Exportation des requêtes en différents formats (cURL, HTTP, etc.)

CONFIGURATION DE SWAGGER UI - APPROCHE BASIQUE (PARTIE 1/3)

La configuration de Swagger UI varie selon le framework utilisé, mais le principe reste identique : exposer la spécification OpenAPI et configurer l'interface utilisateur. Voici un exemple de configuration avec Spring Boot et Springdoc OpenAPI.

CONFIGURATION DE SWAGGER UI - APPROCHE BASIQUE (PARTIE 2/3)

Configuration Maven - Springdoc OpenAPI (xml)

```
1. <!-- Dépendance Maven pour Springdoc OpenAPI -->
2. <dependency>
3.     <groupId>org.springdoc</groupId>
4.     <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
5.     <version>2.3.0</version>
6. </dependency>
```

CONFIGURATION DE SWAGGER UI - APPROCHE BASIQUE (PARTIE 3/3)

Configuration OpenAPI avec Spring Boot (java)

```
1. @Configuration
2. public class OpenAPIConfig {
3.
4.     @Bean
5.     public OpenAPI customOpenAPI() {
6.         return new OpenAPI()
7.             .info(new Info()
8.                 .title("API de Gestion de Produits")
9.                 .version("1.0.0")
10.                .description("API REST pour la gestion de produits e-commerce")
11.                .contact(new Contact()
12.                    .name("Équipe API")
13.                    .email("api@example.com"))
14.                .license(new License()
15.                    .name("Apache 2.0")
16.                    .url("https://www.apache.org/licenses/LICENSE-2.0")));
17.     }
18. }
```

CONFIGURATION AVANCÉE DE SWAGGER UI (PARTIE 1/2)

Configuration avancée dans application.yml (yaml)

```yaml
# Configuration dans application.yml
springdoc:
  api-docs:
    path: /api-docs
    enabled: true
  swagger-ui:
    path: /swagger-ui.html
    enabled: true
    operationsSorter: method
    tagsSorter: alpha
    tryItOutEnabled: true
    filter: true
    displayRequestDuration: true
    defaultModelsExpandDepth: 2
    defaultModelExpandDepth: 2
  show-actuator: false
  paths-to-match: /api/**
  packages-to-scan: com.example.api.controllers
```

CONFIGURATION AVANCÉE DE SWAGGER UI (PARTIE 2/2)

operationsSorter : Tri des opérations par méthode HTTP (GET, POST, PUT, DELETE)

tagsSorter : Tri alphabétique des tags/groupes d'endpoints

tryItOutEnabled : Active par défaut le bouton 'Try it out' pour tester les endpoints

filter : Ajoute une barre de recherche pour filtrer les endpoints

displayRequestDuration : Affiche le temps d'exécution des requêtes

paths-to-match : Limite la documentation aux chemins spécifiés

packages-to-scan : Scanne uniquement les contrôleurs dans les packages définis

PERSONNALISATION DE L'INTERFACE SWAGGER UI (PARTIE 1/3)

Swagger UI peut être personnalisé pour correspondre à l'identité visuelle de votre organisation. La personnalisation inclut le logo, les couleurs, les CSS personnalisés et même l'injection de scripts JavaScript pour des fonctionnalités avancées.

PERSONNALISATION DE L'INTERFACE SWAGGER UI (PARTIE 2/3)

Personnalisation de l'interface Swagger UI (java)

```
 1. @Configuration
 2. public class SwaggerUICustomization implements WebMvcConfigurer {
 3.
 4.     @Override
 5.     public void addResourceHandlers(ResourceHandlerRegistry registry) {
 6.         registry.addResourceHandler("/swagger-ui/**")
 7.                 .addResourceLocations("classpath:/META-INF/
resources/webjars/swagger-ui/")
 8.                 .resourceChain(false);
 9.     }
10.
11.     @Bean
12.     public SwaggerIndexTransformer swaggerIndexTransformer() {
13.         return new SwaggerIndexTransformer() {
14.             @Override
15.             public String transform(String html) {
16.                 return html.replace(
17.                     "<title>Swagger UI</title>",
18.                     "<title>API Documentation - Mon Entreprise</title>"
19.                 ).replace(
20.                     "</head>",
```

PERSONNALISATION DE L'INTERFACE SWAGGER UI (PARTIE 2/3) [A]

Personnalisation de l'interface Swagger UI (java) - Suite

```
21.                 "<link rel='stylesheet' href='/custom-swagger.css'></head>"
22.                    );
23.                 }
24.             };
25.         }
26.  }
```

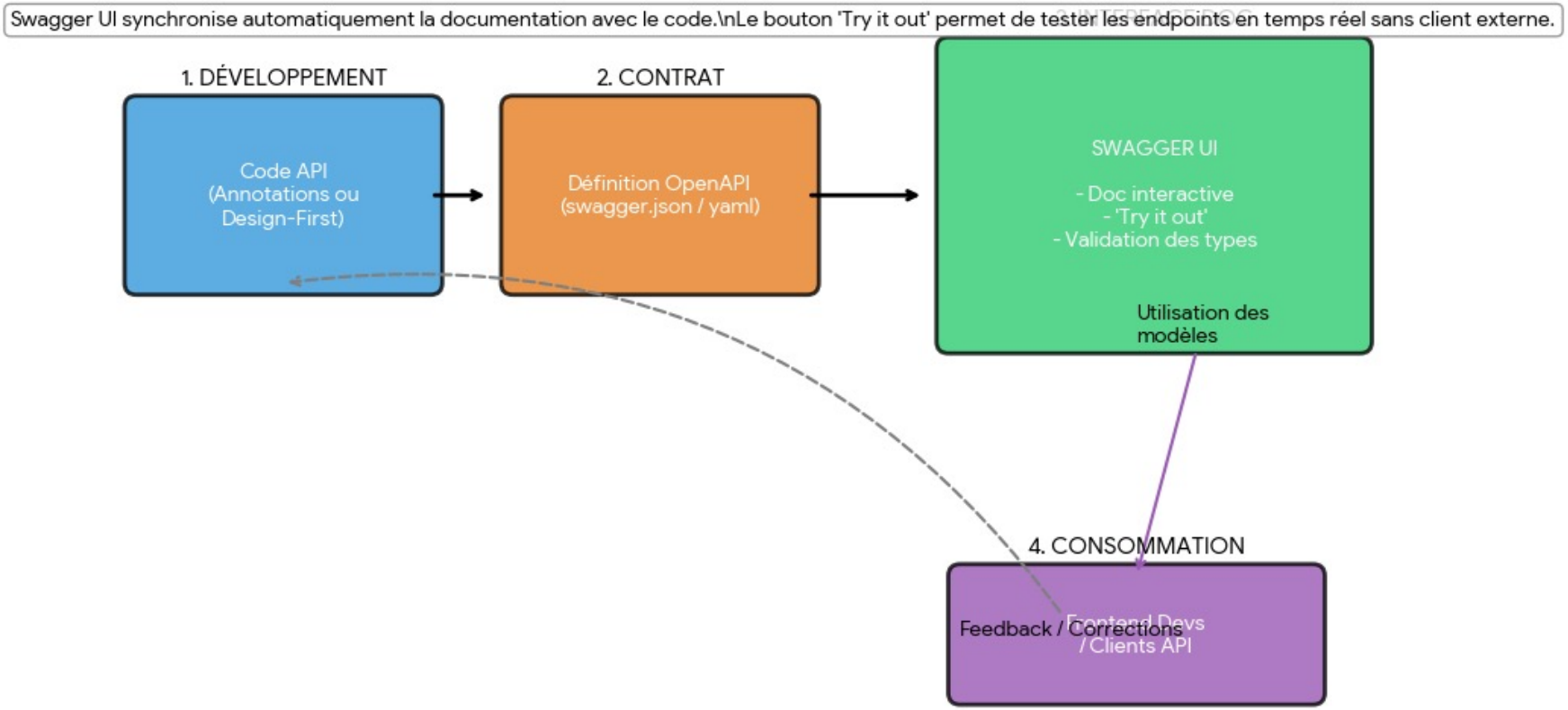
PERSONNALISATION DE L'INTERFACE SWAGGER UI (PARTIE 3/3)

CSS personnalisé pour Swagger UI (css)

```
1. /* Fichier custom-swagger.css dans /static */
2. .swagger-ui .topbar {
3.     background-color: #1e3a8a;
4.     border-bottom: 3px solid #3b82f6;
5. }
6.
7. .swagger-ui .topbar .download-url-wrapper .download-url-button {
8.     background-color: #3b82f6;
9. }
10.
11. .swagger-ui .info .title {
12.     color: #1e3a8a;
13.     font-family: 'Arial', sans-serif;
14. }
15.
16. .swagger-ui .scheme-container {
17.     background-color: #f0f9ff;
18.     border: 1px solid #3b82f6;
19. }
```

INTÉGRATION DE SWAGGER UI DANS LE WORKFLOW DE DÉVELOPPEMENT (PARTIE 1/2)

Workflow : Intégration de Swagger UI dans le Développement



INTÉGRATION DE SWAGGER UI DANS LE WORKFLOW DE DÉVELOPPEMENT (PARTIE 2/2)

Développement piloté par contrat (Contract-First) : La spécification OpenAPI guide l'implémentation

Tests manuels rapides : Les développeurs testent immédiatement leurs endpoints via l'interface

Documentation synchronisée : La documentation est automatiquement mise à jour à chaque modification

Collaboration facilitée : Les équipes frontend peuvent commencer l'intégration en parallèle

Validation continue : Détection précoce des incohérences entre spécification et implémentation

Démos clients : Présentation interactive des fonctionnalités sans client dédié

ENVIRONNEMENTS MULTIPLES ET SÉCURISATION DE SWAGGER UI (PARTIE 1/3)

En production, il est crucial de sécuriser l'accès à Swagger UI ou de le désactiver complètement. La documentation interactive peut exposer des informations sensibles sur la structure de votre API et faciliter les attaques. Voici les stratégies recommandées selon l'environnement.

ENVIRONNEMENTS MULTIPLES ET SÉCURISATION DE SWAGGER UI (PARTIE 2/3)

Stratégies de déploiement Swagger UI par environnement

Environnement	Stratégie	Configuration	Justification
Développement	Activé sans restriction	enabled: true, aucune auth	Facilite les tests et le développement rapide
Staging/QA	Activé avec authentification	enabled: true, Basic Auth ou IP whitelist	Permet les tests d'intégration sécurisés
Production (interne)	Activé avec auth forte	enabled: true, OAuth2 ou JWT requis	Documentation accessible aux partenaires autorisés
Production (publique)	Désactivé ou API publique	enabled: false ou API key publique	Sécurité maximale ou documentation publique intentionnelle

ENVIRONNEMENTS MULTIPLES ET SÉCURISATION DE SWAGGER UI (PARTIE 3/3)

Configuration par environnement avec Spring Profiles (yaml)

```yaml
# Configuration conditionnelle par profil Spring
# application-prod.yml
springdoc:
  swagger-ui:
    enabled: false  # Désactivé en production
  api-docs:
    enabled: false

# application-dev.yml
springdoc:
  swagger-ui:
    enabled: true   # Activé en développement
  api-docs:
    enabled: true
```

VERSIONING D'API REST - ENJEUX ET STRATÉGIES (PARTIE 1/3)

Versioning d'API REST

VERSIONING D'API REST - ENJEUX ET STRATÉGIES (PARTIE 2/3)

Le versioning d'API est essentiel pour faire évoluer votre API sans casser les clients existants. Une stratégie de versioning claire permet d'introduire de nouvelles fonctionnalités, de corriger des erreurs de conception et de déprécier progressivement les anciennes versions tout en maintenant un contrat stable avec les consommateurs.

VERSIONING D'API REST - ENJEUX ET STRATÉGIES (PARTIE 3/3)

Éviter les breaking changes pour les clients existants

Permettre l'évolution progressive de l'API

Donner du temps aux clients pour migrer vers les nouvelles versions

Maintenir plusieurs versions en parallèle pendant une période de transition

Communiquer clairement les changements et les dépréciations

Équilibrer entre innovation et stabilité

STRATÉGIES DE VERSIONING D'API (PARTIE 1/2)

Comparaison des stratégies de versioning d'API REST

Stratégie	Exemple	Avantages	Inconvénients
URI Path	/api/v1/products	Simple, visible, facile à router	Pollution de l'URI, duplication de code
Query Parameter	/api/products?version=1	URI stable, non intrusif	Moins visible, problèmes de cache
Header personnalisé	X-API-Version: 1	URI propre, flexible	Moins découvrable, complexité client
Content Negotiation	Accept: application/vnd.api.v1+json	RESTful pur, sémantique HTTP	Complexe, peu intuitif pour les développeurs
Hostname/Subdomain	v1.api.example.com	Isolation complète, déploiement séparé	Coût infrastructure, gestion DNS

STRATÉGIES DE VERSIONING D'API (PARTIE 2/2)

La stratégie par URI Path (/api/v1/...) est la plus courante et recommandée pour sa simplicité et sa clarté. Elle facilite le routage, la documentation et la compréhension par les développeurs. Les autres approches ont des cas d'usage spécifiques mais sont généralement plus complexes à mettre en œuvre et à maintenir.

IMPLÉMENTATION DU VERSIONING PAR URI PATH (PARTIE 1/2)

Versioning par URI Path avec Spring Boot (java)

```
1. @RestController
2. @RequestMapping("/api/v1/products")
3. public class ProductControllerV1 {
4.
5.     @GetMapping("/{id}")
6.     public ResponseEntity<ProductDtoV1> getProduct(@PathVariable Long id) {
7.         // Implémentation version 1
8.         ProductDtoV1 product = productService.findByIdV1(id);
9.         return ResponseEntity.ok(product);
10.     }
11.
12.     @PostMapping
13.     public ResponseEntity<ProductDtoV1> createProduct(
14.             @Valid @RequestBody ProductDtoV1 product) {
15.         ProductDtoV1 created = productService.createV1(product);
16.         return ResponseEntity.status(HttpStatus.CREATED).body(created);
17.     }
18. }
19.
20. @RestController
```

IMPLÉMENTATION DU VERSIONING PAR URI PATH (PARTIE 1/2) [A]

Versioning par URI Path avec Spring Boot (java) - Suite

```
21. @RequestMapping("/api/v2/products")
22. public class ProductControllerV2 {
23.
24.     @GetMapping("/{id}")
25.     public ResponseEntity<ProductDtoV2> getProduct(@PathVariable Long id) {
26.         // Implémentation version 2 avec champs supplémentaires
27.         ProductDtoV2 product = productService.findByIdV2(id);
28.         return ResponseEntity.ok(product);
29.     }
30.
31.     @PostMapping
32.     public ResponseEntity<ProductDtoV2> createProduct(
33.             @Valid @RequestBody ProductDtoV2 product) {
34.         ProductDtoV2 created = productService.createV2(product);
35.         return ResponseEntity.status(HttpStatus.CREATED).body(created);
36.     }
37. }
```

IMPLÉMENTATION DU VERSIONING PAR URI PATH (PARTIE 2/2)

Créer des contrôleurs séparés pour chaque version majeure

Utiliser des DTOs versionnés (ProductDtoV1, ProductDtoV2) pour éviter les conflits

Maintenir une couche de service qui peut gérer plusieurs versions

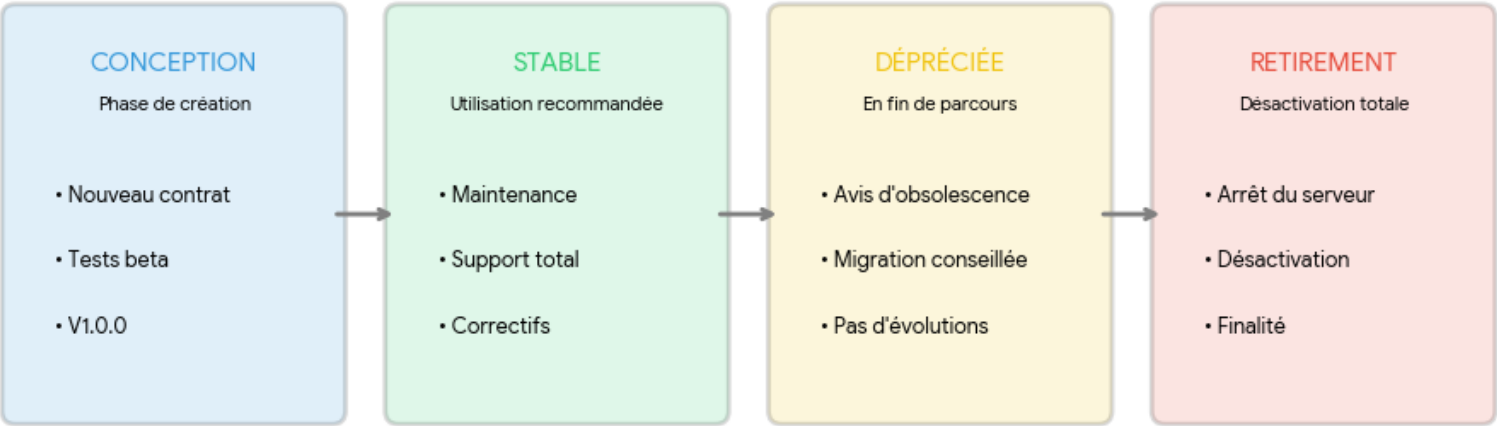
Documenter clairement les différences entre versions dans Swagger

Définir une politique de dépréciation (ex: support de N-1 versions)

Utiliser @Deprecated pour marquer les anciennes versions

GESTION DU CYCLE DE VIE DES VERSIONS (PARTIE 1/3)

Gestion du Cycle de Vie des Versions d'une API



Le versioning permet de faire coexister plusieurs états d'une API.\nOn utilise généralement le Semantic Versioning (v1, v2) ou des dates.

GESTION DU CYCLE DE VIE DES VERSIONS (PARTIE 2/3)

Signalisation de dépréciation via headers HTTP (java)

```
1. @RestController
2. @RequestMapping("/api/v1/products")
3. @Deprecated(since = "2.0", forRemoval = true)
4. public class ProductControllerV1 {
5.
6.     @GetMapping("/{id}")
7.     public ResponseEntity<ProductDtoV1> getProduct(@PathVariable Long id) {
8.         // Ajouter un header de dépréciation
9.         HttpHeaders headers = new HttpHeaders();
10.        headers.add("Deprecation", "true");
11.        headers.add("Sunset", "2024-12-31");
12.        headers.add("Link", "</api/v2/products>; rel=\"successor-version\"");
13.
14.        ProductDtoV1 product = productService.findByIdV1(id);
15.        return ResponseEntity.ok().headers(headers).body(product);
16.    }
17. }
```

GESTION DU CYCLE DE VIE DES VERSIONS (PARTIE 3/3)

Annoncer la dépréciation au moins 6-12 mois à l'avance

Utiliser les headers HTTP standardisés (Deprecation, Sunset, Link)

Documenter le calendrier de dépréciation dans la documentation

Envoyer des notifications aux clients enregistrés

Fournir un guide de migration détaillé vers la nouvelle version

Maintenir des métriques d'utilisation pour planifier le retrait

Offrir un support étendu pour les clients critiques

GESTION DES ERREURS STANDARDISÉES (PARTIE 1/3)

Gestion des Erreurs Standardisées

GESTION DES ERREURS STANDARDISÉES (PARTIE 2/3)

Une gestion des erreurs cohérente et standardisée est cruciale pour une API REST professionnelle. Elle permet aux clients de comprendre rapidement ce qui s'est mal passé, de gérer les erreurs de manière programmatique et d'améliorer l'expérience développeur. Un format d'erreur structuré et documenté facilite le débogage et réduit le temps d'intégration.

GESTION DES ERREURS STANDARDISÉES (PARTIE 3/3)

Utiliser les codes de statut HTTP appropriés (4xx pour erreurs client, 5xx pour erreurs serveur)

Fournir un format d'erreur JSON cohérent sur toute l'API

Inclure un code d'erreur applicatif pour un traitement programmatique

Donner des messages d'erreur clairs et actionnables

Ne jamais exposer de détails techniques sensibles (stack traces, requêtes SQL)

Inclure un identifiant de corrélation pour le traçage

Documenter toutes les erreurs possibles dans la spécification OpenAPI

FORMAT D'ERREUR STANDARDISÉ - RFC 7807 (PARTIE 1/3)

Le RFC 7807 définit un format standard 'Problem Details' pour les erreurs HTTP. Ce format JSON structuré est adopté par de nombreuses organisations et frameworks. Il fournit un cadre extensible pour communiquer les détails d'erreur de manière cohérente.

FORMAT D'ERREUR STANDARDISÉ - RFC 7807 (PARTIE 2/3)

Exemple de réponse d'erreur selon RFC 7807 (json)

```
 1. {
 2.   "type": "https://api.example.com/errors/insufficient-stock",
 3.   "title": "Stock insuffisant",
 4.   "status": 400,
 5.   "detail": "Le produit 'iPhone 15 Pro' n'a que 2 unités en stock,
mais vous en avez demandé 5",
 6.   "instance": "/api/v1/orders/12345",
 7.   "timestamp": "2024-01-15T14:30:00Z",
 8.   "correlationId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
 9.   "errors": [
10.     {
11.       "field": "items[0].quantity",
12.       "code": "STOCK_INSUFFICIENT",
13.       "message": "Quantité demandée (5) supérieure au stock disponible (2)",
14.       "availableStock": 2,
15.       "requestedQuantity": 5
16.     }
17.   ]
18. }
```

FORMAT D'ERREUR STANDARDISÉ - RFC 7807 (PARTIE 3/3)

Champs du format Problem Details (RFC 7807)

Champ	Type	Obligatoire	Description
type	URI	Non	URI identifiant le type d'erreur (documentation)
title	String	Non	Résumé court et lisible du problème
status	Integer	Non	Code de statut HTTP (doit correspondre à la réponse)
detail	String	Non	Explication détaillée spécifique à cette occurrence
instance	URI	Non	URI identifiant l'occurrence spécifique du problème
timestamp	DateTime	Extension	Horodatage de l'erreur (extension courante)
correlationId	UUID	Extension	ID de corrélation pour le traçage (extension)
errors	Array	Extension	Liste détaillée des erreurs de validation (extension)

IMPLÉMENTATION DE LA GESTION D'ERREURS GLOBALE

Exemple de Code (Code)

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiError> handleAllExceptions(Exception ex,
 HttpServletRequest request) {

        ApiError error = ApiError.builder()
                .type("https://monsite.com")
                .title("Une erreur interne est survenue")
                .status(HttpStatus.INTERNAL_SERVER_ERROR.value())
                .detail(ex.getMessage())
                .instance(request.getRequestURI())
                .timestamp(Instant.now())
                .correlationId(UUID.randomUUID().toString())
// Utile pour retrouver l'erreur dans les logs
                .build();

        return new ResponseEntity<>(error, HttpStatus.INTERNAL_SERVER_ERROR);
    }

}
```

LECTURE 4 : TECHNOLOGIES ALTERNATIVES ET ARCHITECTURES ÉVÉNEMENTIELLES MODERNES

- GraphQL : principes fondamentaux, schémas, requêtes flexibles, mutations et comparaison avec REST pour les cas d'usage appropriés, - gRPC et Protocol Buffers : communication haute performance, strea

INTRODUCTION AUX TECHNOLOGIES ALTERNATIVES (PARTIE 1/2)

Bien que REST soit devenu le standard de facto pour les API web, plusieurs technologies alternatives ont émergé pour répondre à des besoins spécifiques que REST ne couvre pas optimalement. GraphQL offre une flexibilité accrue dans les requêtes de données, gRPC privilégie la performance et l'efficacité pour les communications inter-services, tandis que les Webhooks permettent des architectures événementielles réactives.

INTRODUCTION AUX TECHNOLOGIES ALTERNATIVES (PARTIE 2/2)

Limitations de REST nécessitant des alternatives :

- Over-fetching et under-fetching de données
- Performance limitée pour les communications microservices
- Modèle pull uniquement (polling nécessaire pour les mises à jour)
- Verbosité du format JSON pour les échanges haute fréquence

Critères de choix technologique :

- Nature des données et patterns d'accès
- Exigences de performance et latence
- Architecture système (monolithique vs microservices)
- Besoins de réactivité en temps réel

GRAPHQL : PRINCIPES FONDAMENTAUX (PARTIE 1/3)

GraphQL est un langage de requête pour API et un runtime pour exécuter ces requêtes, développé par Facebook en 2012 et open-sourcé en 2015. Contrairement à REST qui expose plusieurs endpoints fixes, GraphQL expose un seul endpoint et permet au client de spécifier exactement les données dont il a besoin via une requête déclarative.

GRAPHQL : PRINCIPES FONDAMENTAUX (PARTIE 2/3)

Concepts fondamentaux :

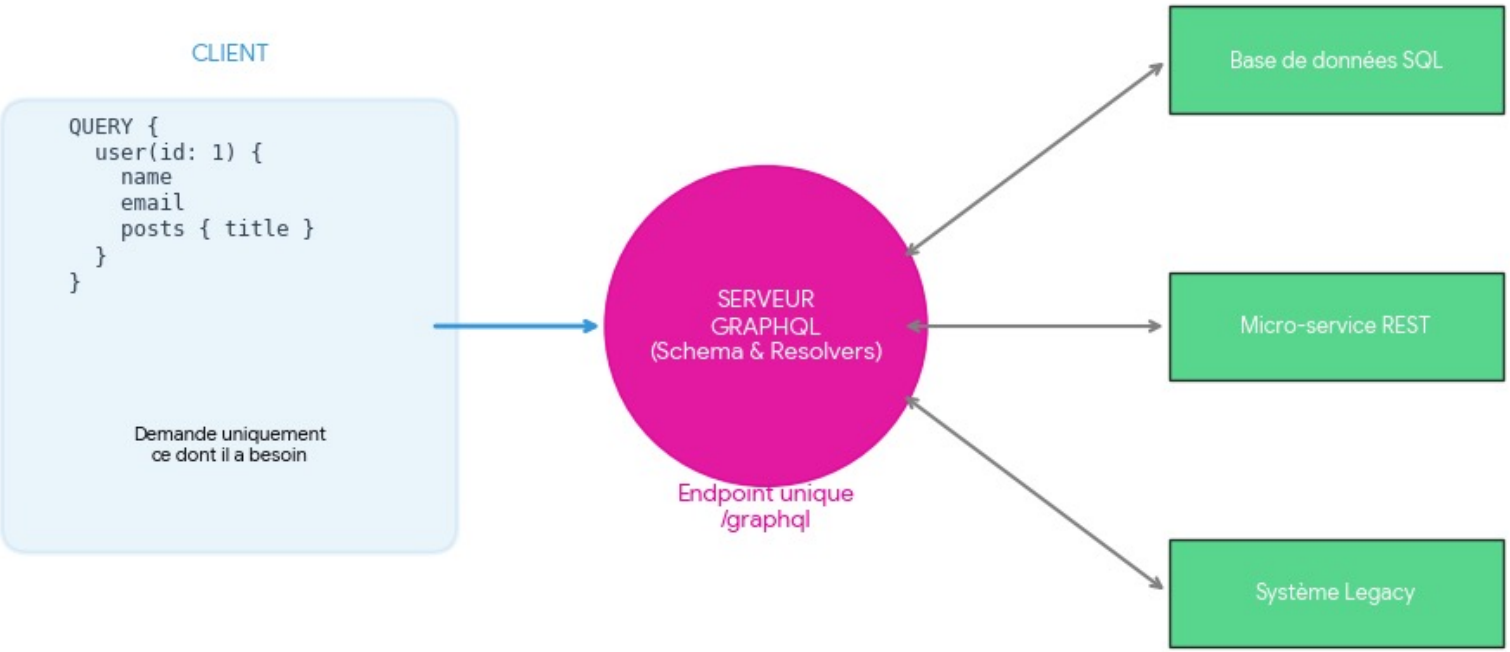
- Schéma typé : définition stricte des types de données disponibles
- Requêtes déclaratives : le client demande exactement ce dont il a besoin
- Résolution hiérarchique : navigation dans le graphe de données
- Introspection : le schéma est auto-documenté et interrogeable

Avantages principaux :

- Élimination de l'over-fetching et under-fetching
- Réduction du nombre de requêtes réseau
- Évolution de l'API sans versioning
- Typage fort et validation automatique

GRAPHQL : PRINCIPES FONDAMENTAUX (PARTIE 3/3)

GraphQL : Architecture et Principes Fondamentaux



SCHÉMAS GRAPHQL : DÉFINITION DES TYPES (PARTIE 1/3)

Le schéma GraphQL définit le contrat entre le client et le serveur en utilisant le Schema Definition Language (SDL). Il spécifie les types de données disponibles, leurs champs, les relations entre types et les opérations possibles (Query, Mutation, Subscription).

SCHÉMAS GRAPHQL : DÉFINITION DES TYPES (PARTIE 2/3)

Exemple de schéma GraphQL pour un blog (graphql)

```
# Définition de types dans un schéma GraphQL
type User {
  id: ID!
  username: String!
  email: String!
  posts: [Post!]!
  createdAt: DateTime!
}

type Post {
  id: ID!
  title: String!
  content: String!
  author: User!
  comments: [Comment!]!
  publishedAt: DateTime
}

type Comment {
  id: ID!
```

SCHÉMAS GRAPHQL : DÉFINITION DES TYPES (PARTIE 2/3) [A]

Exemple de schéma GraphQL pour un blog (graphql) - Suite

```
21.   text: String!
22.   author: User!
23.   post: Post!
24. }
25.
26. # Point d'entrée pour les requêtes
27. type Query {
28.   user(id: ID!): User
29.   users(limit: Int, offset: Int): [User!]!
30.   post(id: ID!): Post
31.   posts(authorId: ID): [Post!]!
32. }
33.
34. # Point d'entrée pour les mutations
35. type Mutation {
36.   createUser(username: String!, email: String!): User!
37.   createPost(title: String!, content: String!, authorId: ID!): Post!
38.   deletePost(id: ID!): Boolean!
39. }
```

SCHÉMAS GRAPHQL : DÉFINITION DES TYPES (PARTIE 3/3)

Types scalaires de base :

- Int : entier signé 32-bit
- Float : nombre à virgule flottante
- String : chaîne de caractères UTF-8
- Boolean : true ou false
- ID : identifiant unique sérialisé en string

Modificateurs de types :

- ! : champ non-nullable (obligatoire)
- [Type] : liste d'éléments
- [Type!]! : liste non-null contenant des éléments non-null

REQUÊTES GRAPHQL : FLEXIBILITÉ ET PRÉCISION (PARTIE 1/3)

Les requêtes GraphQL permettent au client de spécifier exactement la structure des données souhaitées. Le serveur retourne une réponse JSON qui correspond exactement à la forme de la requête, éliminant ainsi le sur-fetching et le sous-fetching de données.

REQUÊTES GRAPHQL : FLEXIBILITÉ ET PRÉCISION (PARTIE 2/3)

Exemples de requêtes GraphQL (graphql)

```
# Requête simple pour obtenir un utilisateur
query {
  user(id: "123") {
    username
    email
  }
}

# Requête avec navigation dans les relations
query {
  user(id: "123") {
    username
    posts {
      title
      publishedAt
      comments {
        text
        author {
          username
        }
```

REQUÊTES GRAPHQL : FLEXIBILITÉ ET PRÉCISION (PARTIE 2/3) [A]

Exemples de requêtes GraphQL (graphql) - Suite

```
21.         }
22.       }
23.     }
24. }
25.
26. # Requête avec alias et fragments
27. query {
28.   mainAuthor: user(id: "123") {
29.     ...userFields
30.   }
31.   secondaryAuthor: user(id: "456") {
32.     ...userFields
33.   }
34. }
35.
36. fragment userFields on User {
37.   username
38.   email
39.   posts {
40.     title
```

REQUÊTES GRAPHQL : FLEXIBILITÉ ET PRÉCISION (PARTIE 2/3) [B]

Exemples de requêtes GraphQL (graphql) - Suite

```
41.   }
42. }
43.
44. # Requête avec variables
45. query GetUserPosts($userId: ID!, $limit: Int) {
46.   user(id: $userId) {
47.     username
48.     posts(limit: $limit) {
49.       title
50.       content
51.     }
52.   }
53. }
```

REQUÊTES GRAPHQL : FLEXIBILITÉ ET PRÉCISION (PARTIE 3/3)

Fonctionnalités avancées des requêtes :

- Alias : renommer les champs dans la réponse
- Fragments : réutiliser des sélections de champs
- Variables : paramétrer les requêtes dynamiquement
- Directives : modifier l'exécution (@include, @skip)
- Requêtes multiples : combiner plusieurs requêtes en une seule

MUTATIONS GRAPHQL : MODIFICATION DES DONNÉES (PARTIE 1/3)

Les mutations GraphQL sont utilisées pour modifier les données côté serveur (création, mise à jour, suppression). Contrairement aux requêtes qui peuvent être exécutées en parallèle, les mutations sont exécutées séquentiellement pour garantir la cohérence des données.

MUTATIONS GRAPHQL : MODIFICATION DES DONNÉES (PARTIE 2/3)

Exemples de mutations GraphQL (graphql)

```
# Mutation de création avec retour de l'objet créé
mutation {
  createPost(
    title: "Introduction à GraphQL"
    content: "GraphQL est un langage de requête..."
    authorId: "123"
  ) {
    id
    title
    author {
      username
    }
    createdAt
  }
}

# Mutation de mise à jour
mutation {
  updatePost(
    id: "789"
```

MUTATIONS GRAPHQL : MODIFICATION DES DONNÉES (PARTIE 2/3) [A]

Exemples de mutations GraphQL (graphql) - Suite

```
21.     title: "GraphQL : Guide Complet"
22.   ) {
23.     id
24.     title
25.     updatedAt
26.   }
27. }
28.
29. # Mutation de suppression
30. mutation {
31.   deletePost(id: "789")
32. }
33.
34. # Mutations multiples séquentielles
35. mutation {
36.   first: createUser(username: "alice", email: "alice@example.com") {
37.     id
38.   }
39.   second: createPost(
40.     title: "Premier post"
```

MUTATIONS GRAPHQL : MODIFICATION DES DONNÉES (PARTIE 2/3) [B]

Exemples de mutations GraphQL (graphql) - Suite

```
41.     content: "Contenu..."
42.     authorId: "123"
43.   ) {
44.     id
45.   }
46. }
47.
48. # Mutation avec variables et input type
49. mutation CreatePost($input: CreatePostInput!) {
50.   createPost(input: $input) {
51.     id
52.     title
53.     author {
54.       username
55.     }
56.   }
57. }
58.
59. # Variables:
60. # {
```

MUTATIONS GRAPHQL : MODIFICATION DES DONNÉES (PARTIE 2/3) [C]

Exemples de mutations GraphQL (graphql) - Suite

```
61. #   "input": {
62. #     "title": "Mon article",
63. #     "content": "Contenu de l'article",
64. #     "authorId": "123"
65. #   }
66. # }
```

MUTATIONS GRAPHQL : MODIFICATION DES DONNÉES (PARTIE 3/3)

Bonnes pratiques pour les mutations :

- Retourner l'objet modifié pour mise à jour du cache client
- Utiliser des Input Types pour grouper les paramètres
- Implémenter une gestion d'erreurs granulaire
- Garantir l'idempotence quand possible
- Valider les données côté serveur avant modification

GRAPHQL VS REST : COMPARAISON DÉTAILLÉE (PARTIE 1/2)

Comparaison GraphQL vs REST

Critère	REST	GraphQL
Endpoints	Multiples endpoints par ressource	Endpoint unique /graphql
Récupération données	Données fixes par endpoint	Client spécifie exactement ses besoins
Over-fetching	Fréquent (trop de données)	Éliminé (requête précise)
Under-fetching	Nécessite requêtes multiples	Une requête pour données relationnelles
Versioning	Versioning d'API (v1, v2)	Évolution sans versioning
Typage	Optionnel (OpenAPI)	Typage fort obligatoire
Cache HTTP	Cache natif (GET)	Cache complexe (POST)
Courbe apprentissage	Simple et familier	Plus complexe initialement
Monitoring	Logs par endpoint	Requêtes variables, monitoring complexe
Performance	Optimisé pour cas simples	Optimisé pour requêtes complexes

GRAPHQL VS REST : COMPARAISON DÉTAILLÉE (PARTIE 2/2)

Cas d'usage appropriés pour GraphQL :

- Applications avec besoins de données variés (mobile, web, desktop)
- Interfaces riches nécessitant des données relationnelles complexes
- Équipes frontend/backend découplées
- Évolution rapide des besoins en données

Cas où REST reste préférable :

- APIs publiques simples et stables
- Besoin de cache HTTP standard
- Opérations CRUD simples
- Équipe sans expertise GraphQL

IMPLÉMENTATION GRAPHQL : EXEMPLE PRATIQUE (PARTIE 1/2)

Serveur GraphQL avec Apollo Server (javascript)

```
1. // Serveur GraphQL avec Apollo Server (Node.js)
2. const { ApolloServer, gql } = require('apollo-server');
3.
4. // Définition du schéma
5. const typeDefs = gql`
6.   type User {
7.     id: ID!
8.     username: String!
9.     email: String!
10.    posts: [Post!]!
11.  }
12.
13.  type Post {
14.    id: ID!
15.    title: String!
16.    content: String!
17.    author: User!
18.  }
19.
20.  type Query {
```

IMPLÉMENTATION GRAPHQL : EXEMPLE PRATIQUE (PARTIE 1/2) [A]

Serveur GraphQL avec Apollo Server (javascript) - Suite

```
21.     users: [User!]!
22.     user(id: ID!): User
23.     posts: [Post!]!
24.   }
25.
26.   type Mutation {
27.     createPost(title: String!, content: String!, authorId: ID!): Post!
28.   }
29. `;
30.
31. // Resolvers : logique de résolution des données
32. const resolvers = {
33.   Query: {
34.     users: () => getUsersFromDB(),
35.     user: (parent, { id }) => getUserById(id),
36.     posts: () => getPostsFromDB(),
37.   },
38.   Mutation: {
39.     createPost: (parent, { title, content, authorId }) => {
40.       return createPostInDB({ title, content, authorId });
```

IMPLÉMENTATION GRAPHQL : EXEMPLE PRATIQUE (PARTIE 1/2) [B]

Serveur GraphQL avec Apollo Server (javascript) - Suite

```
41.     },
42.   },
43.   User: {
44.     // Résolution des relations
45.     posts: (user) => getPostsByAuthorId(user.id),
46.   },
47.   Post: {
48.     author: (post) => getUserById(post.authorId),
49.   },
50. };
51.
52. // Création du serveur
53. const server = new ApolloServer({
54.   typeDefs,
55.   resolvers,
56.   context: ({ req }) => ({
57.     // Context partagé (auth, dataloaders, etc.)
58.     user: authenticateUser(req),
59.   }),
60. });
```

IMPLÉMENTATION GRAPHQL : EXEMPLE PRATIQUE (PARTIE 1/2) [C]

Serveur GraphQL avec Apollo Server (javascript) - Suite

```
61.
62. server.listen().then(({ url }) => {
63.   console.log(`Server ready at ${url}`);
64. });
```

IMPLÉMENTATION GRAPHQL : EXEMPLE PRATIQUE (PARTIE 2/2)

Composants clés d'un serveur GraphQL :

- Type Definitions : schéma SDL définissant les types
- Resolvers : fonctions résolvant chaque champ
- Context : données partagées (authentification, DB)
- DataLoaders : optimisation requêtes (batching, cache)

Outils et frameworks populaires :

- Apollo Server (Node.js) : solution complète et mature
- GraphQL Yoga : serveur léger et flexible
- Hasura : génération automatique depuis base de données
- Prisma : ORM avec génération GraphQL

GRPC : COMMUNICATION HAUTE PERFORMANCE (PARTIE 1/3)

gRPC (gRPC Remote Procedure Call) est un framework RPC open-source développé par Google, utilisant HTTP/2 comme protocole de transport et Protocol Buffers comme format de sérialisation. Il est conçu pour des communications inter-services hautement performantes, particulièrement dans les architectures microservices.

GRPC : COMMUNICATION HAUTE PERFORMANCE (PARTIE 2/3)

Caractéristiques principales de gRPC :

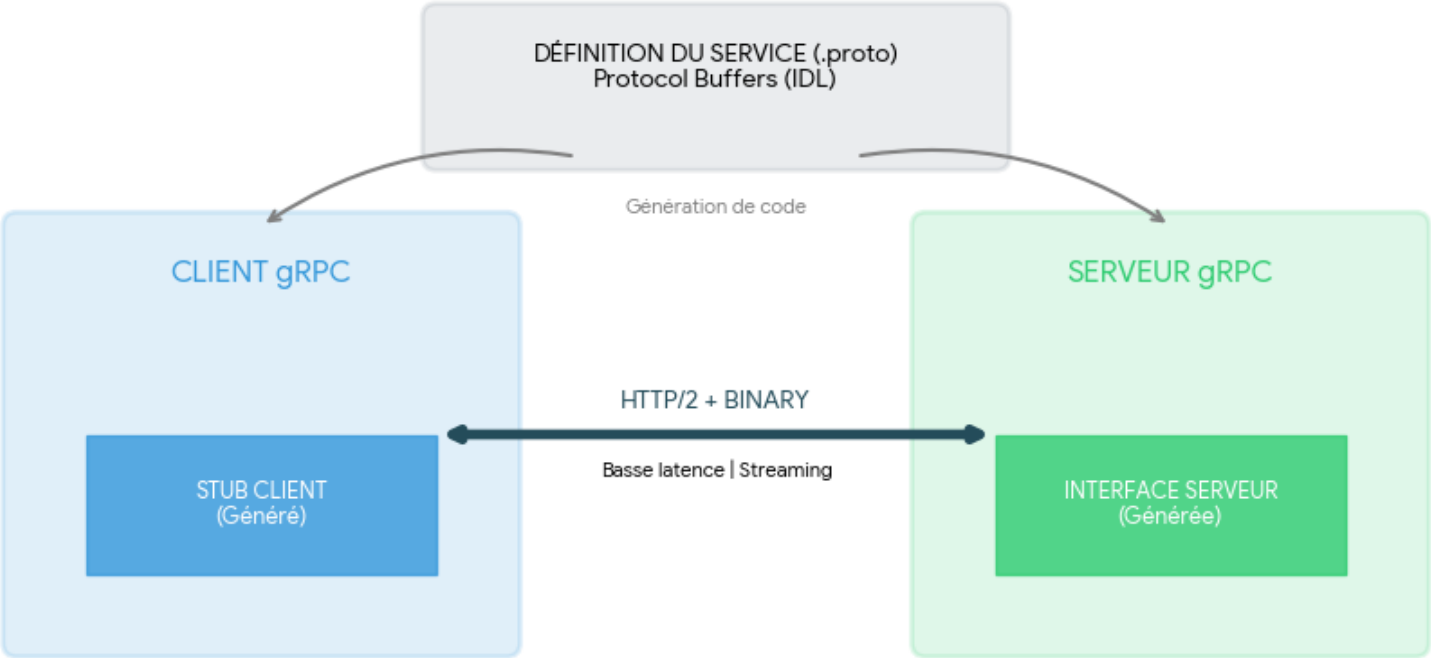
- Protocol Buffers : sérialisation binaire compacte et rapide
- HTTP/2 : multiplexing, compression headers, streaming bidirectionnel
- Génération de code : clients et serveurs générés automatiquement
- Support multi-langages : 10+ langages officiellement supportés
- Streaming : unaire, server-streaming, client-streaming, bidirectionnel

Avantages pour les microservices :

- Performance : 5-10x plus rapide que REST/JSON
- Typage fort : contrat strict via .proto files
- Efficacité réseau : payload réduit de 30-50%
- Load balancing et service discovery intégrés

GRPC : COMMUNICATION HAUTE PERFORMANCE (PARTIE 3/3)

gRPC : Architecture et Communication Haute Performance



PROTOCOL BUFFERS : SÉRIALISATION EFFICACE (PARTIE 1/3)

Protocol Buffers (protobuf) est un mécanisme de sérialisation de données structurées, neutre en termes de langage et de plateforme. Il permet de définir la structure des données une seule fois dans un fichier .proto, puis de générer automatiquement du code pour lire et écrire ces données dans différents langages.

PROTOCOL BUFFERS : SÉRIALISATION EFFICACE (PARTIE 2/3)

Définition de service gRPC avec Protocol Buffers (protobuf)

```
1. // Définition d'un service gRPC avec Protocol Buffers
2. syntax = "proto3";
3.
4. package blog;
5.
6. // Définition des messages (structures de données)
7. message User {
8.   int32 id = 1;
9.   string username = 2;
10.   string email = 3;
11.   repeated Post posts = 4;
12. }
13.
14. message Post {
15.   int32 id = 1;
16.   string title = 2;
17.   string content = 3;
18.   int32 author_id = 4;
19.   google.protobuf.Timestamp created_at = 5;
20. }
```

PROTOCOL BUFFERS : SÉRIALISATION EFFICACE (PARTIE 2/3) [A]

Définition de service gRPC avec Protocol Buffers (protobuf) - Suite

```
21.
22. message GetUserRequest {
23.   int32 user_id = 1;
24. }
25.
26. message GetUserResponse {
27.   User user = 1;
28. }
29.
30. message CreatePostRequest {
31.   string title = 1;
32.   string content = 2;
33.   int32 author_id = 3;
34. }
35.
36. message ListPostsRequest {
37.   int32 author_id = 1;
38.   int32 page_size = 2;
39.   string page_token = 3;
40. }
```

PROTOCOL BUFFERS : SÉRIALISATION EFFICACE (PARTIE 2/3) [B]

Définition de service gRPC avec Protocol Buffers (protobuf) - Suite

```
41.
42. message ListPostsResponse {
43.   repeated Post posts = 1;
44.   string next_page_token = 2;
45. }
46.
47. // Définition du service
48. service BlogService {
49.   // RPC unaire
50.   rpc GetUser(GetUserRequest) returns (GetUserResponse);
51.
52.   // RPC avec server streaming
53.   rpc ListPosts(ListPostsRequest) returns (stream Post);
54.
55.   // RPC unaire
56.   rpc CreatePost(CreatePostRequest) returns (Post);
57.
58.   // RPC avec client streaming
59.   rpc CreatePosts(stream CreatePostRequest) returns (ListPostsResponse);
60.
```

PROTOCOL BUFFERS : SÉRIALISATION EFFICACE (PARTIE 2/3) [C]

Définition de service gRPC avec Protocol Buffers (protobuf) - Suite

```
61.   // RPC bidirectionnel
62.   rpc Chat(stream ChatMessage) returns (stream ChatMessage);
63. }
```

PROTOCOL BUFFERS : SÉRIALISATION EFFICACE (PARTIE 3/3)

Types de données Protocol Buffers :

- Scalaires : int32, int64, float, double, bool, string, bytes
- Énumérations : enum pour valeurs prédéfinies
- Messages imbriqués : composition de structures
- Repeated : équivalent de tableaux/listes
- Maps : associations clé-valeur

Avantages de Protocol Buffers :

- Compact : 3-10x plus petit que JSON
- Rapide : parsing 20-100x plus rapide que JSON
- Évolutif : compatibilité ascendante/descendante
- Validé : typage fort avec validation automatique

MODES DE STREAMING GRPC (PARTIE 1/3)

gRPC supporte quatre modes de communication grâce à HTTP/2, permettant des patterns d'échange de données variés et optimisés selon les besoins. Ces modes offrent une flexibilité bien supérieure à REST qui est limité au pattern requête-réponse.

MODES DE STREAMING GRPC (PARTIE 2/3)

Modes de streaming gRPC

Mode	Description	Cas d'usage	Exemple
Unaire	1 requête → 1 réponse	Opérations CRUD classiques	GetUser, CreatePost
Server Streaming	1 requête → N réponses	Téléchargement de données volumineuses, notifications	ListPosts, StreamLogs
Client Streaming	N requêtes → 1 réponse	Upload de fichiers, agrégation de données	UploadFile, BatchInsert
Bidirectionnel	N requêtes ↔️ N réponses	Chat temps réel, collaboration	ChatRoom, LiveUpdates

MODES DE STREAMING GRPC (PARTIE 3/3)

Implémentation des modes de streaming gRPC (javascript)

```
 1. // Implémentation serveur gRPC avec streaming (Node.js)
 2. const grpc = require('@grpc/grpc-js');
 3. const protoLoader = require('@grpc/proto-loader');
 4.
 5. // Chargement du fichier .proto
 6. const packageDefinition = protoLoader.loadSync('blog.proto');
 7. const blogProto = grpc.loadPackageDefinition(packageDefinition).blog;
 8.
 9. // Implémentation des méthodes du service
10. const blogService = {
11.   // RPC Unaire
12.   getUser: (call, callback) => {
13.     const userId = call.request.user_id;
14.     const user = getUserFromDB(userId);
15.     callback(null, { user });
16.   },
17.
18.   // Server Streaming : envoie plusieurs réponses
19.   listPosts: (call) => {
20.     const authorId = call.request.author_id;
```

MODES DE STREAMING GRPC (PARTIE 3/3) [A]

Implémentation des modes de streaming gRPC (javascript) - Suite

```
21.     const posts = getPostsByAuthor(authorId);
22.
23.     posts.forEach(post => {
24.       call.write(post); // Envoie chaque post séparément
25.     });
26.     call.end(); // Fin du stream
27.   },
28.
29.   // Client Streaming : reçoit plusieurs requêtes
30.   createPosts: (call, callback) => {
31.     const posts = [];
32.
33.     call.on('data', (request) => {
34.       // Reçoit chaque post du client
35.       const post = createPostInDB(request);
36.       posts.push(post);
37.     });
38.
39.     call.on('end', () => {
40.       // Envoie la réponse finale
```

MODES DE STREAMING GRPC (PARTIE 3/3) [B]

Implémentation des modes de streaming gRPC (javascript) - Suite

```
41.         callback(null, { posts });
42.       });
43.     },
44.
45.     // Bidirectionnel : envoie et reçoit en parallèle
46.     chat: (call) => {
47.       call.on('data', (message) => {
48.         console.log('Received:', message.text);
49.         // Broadcast à tous les clients
50.         call.write({ text: `Echo: ${message.text}` });
51.       });
52.
53.       call.on('end', () => {
54.         call.end();
55.       });
56.     },
57. };
58.
59. // Démarrage du serveur
60. const server = new grpc.Server();
```

MODES DE STREAMING GRPC (PARTIE 3/3) [C]

Implémentation des modes de streaming gRPC (javascript) - Suite

```
61. server.addService(blogProto.BlogService.service, blogService);
62. server.bindAsync(
63.   '0.0.0.0:50051',
64.   grpc.ServerCredentials.createInsecure(),
65.   () => server.start()
66. );
```

COMPARAISON DES PARADIGMES : REST VS GRAPHQL VS GRPC (PARTIE 1/2)

Chaque paradigme d'API répond à des besoins différents et présente des caractéristiques distinctes. REST domine pour sa simplicité et sa standardisation, GraphQL offre une flexibilité maximale pour les clients, tandis que gRPC privilégie la performance et l'efficacité dans les communications inter-services.

COMPARAISON DES PARADIGMES : REST VS GRAPHQL VS GRPC (PARTIE 2/2)

REST : Architecture basée sur les ressources et HTTP standard

- Communication stateless avec méthodes HTTP
- Réponses souvent en JSON ou XML
- Documentation via OpenAPI/Swagger

GraphQL : Langage de requête flexible pour les API

- Le client spécifie exactement les données nécessaires
- Une seule endpoint pour toutes les opérations
- Résout les problèmes d'over-fetching et under-fetching

gRPC : Framework RPC haute performance de Google

- Utilise Protocol Buffers pour la sérialisation
- Communication binaire sur HTTP/2
- Support natif du streaming bidirectionnel

TABLEAU COMPARATIF DÉTAILLÉ DES PARADIGMES

Comparaison REST vs GraphQL vs gRPC

Critère	REST	GraphQL	gRPC
Protocole	HTTP/1.1	HTTP/1.1 ou HTTP/2	HTTP/2
Format de données	JSON, XML	JSON	Protocol Buffers (binaire)
Endpoints	Multiples (/users, /posts)	Unique (/graphql)	Définis par services
Typage	Faible (documentation)	Fort (schéma GraphQL)	Très fort (protobuf)
Versioning	Via URL ou headers	Évolution du schéma	Compatibilité protobuf
Caching	Natif HTTP	Complexe	Non natif
Courbe d'apprentissage	Faible	Moyenne	Élevée
Performance	Bonne	Moyenne	Excellente
Streaming	Non natif (SSE)	Subscriptions	Natif bidirectionnel

EXEMPLES DE REQUÊTES : REST (PARTIE 1/2)

Requêtes REST classiques (http)

```
# Récupération d'un utilisateur et ses posts en REST
# Nécessite plusieurs appels (N+1 problem potentiel)

# Requête 1 : Obtenir l'utilisateur
GET /api/users/123
Host: api.example.com
Accept: application/json

# Réponse 1
{
  "id": 123,
  "name": "Alice Dupont",
  "email": "alice@example.com",
  "posts_url": "/api/users/123/posts"
}

# Requête 2 : Obtenir les posts de l'utilisateur
GET /api/users/123/posts
Host: api.example.com

```

EXEMPLES DE REQUÊTES : REST (PARTIE 1/2) [A]

Requêtes REST classiques (http) - Suite

```
21. # Réponse 2
22. [
23.   {"id": 1, "title": "Mon premier post", "content": "..."},
24.   {"id": 2, "title": "Deuxième article", "content": "..."}
25. ]
```

EXEMPLES DE REQUÊTES : REST (PARTIE 2/2)

REST nécessite souvent plusieurs requêtes pour récupérer des données relationnelles, ce qui peut entraîner des problèmes de performance (N+1 queries). Cependant, chaque endpoint est simple, prévisible et bénéficie du caching HTTP standard.

EXEMPLES DE REQUÊTES : GRAPHQL (PARTIE 1/2)

Requête GraphQL unique et flexible (graphql)

```
1. # Même besoin en GraphQL : une seule requête
2.
3. POST /graphql
4. Content-Type: application/json
5.
6. # Corps de la requête
7. {
8.   "query": """
9.     query GetUserWithPosts($userId: ID!) {
10.       user(id: $userId) {
11.         id
12.         name
13.         email
14.         posts {
15.           id
16.           title
17.           content
18.           createdAt
19.         }
20.       }
```

EXEMPLES DE REQUÊTES : GRAPHQL (PARTIE 1/2) [A]

Requête GraphQL unique et flexible (graphql) - Suite

```
21.     }
22.   """,
23.   "variables": {
24.     "userId": "123"
25.   }
26. }
27.
28. # Réponse unique et structurée
29. {
30.   "data": {
31.     "user": {
32.       "id": "123",
33.       "name": "Alice Dupont",
34.       "email": "alice@example.com",
35.       "posts": [
36.         {"id": "1", "title": "Mon premier post", "content": "...", "
createdAt": "2024-01-15"},
37.         {"id": "2", "title": "Deuxième article", "content": "...",
 "createdAt": "2024-01-20"}
38.       ]
39.     }
40.   }
```

EXEMPLES DE REQUÊTES : GRAPHQL (PARTIE 1/2) [B]

Requête GraphQL unique et flexible (graphql) - Suite

```
41. }
```

EXEMPLES DE REQUÊTES : GRAPHQL (PARTIE 2/2)

GraphQL permet de récupérer exactement les données nécessaires en une seule requête. Le client spécifie la structure souhaitée, évitant l'over-fetching (trop de données) et l'under-fetching (pas assez de données). Le schéma GraphQL garantit le typage fort et l'auto-documentation.

EXEMPLES DE REQUÊTES : GRPC (PARTIE 1/2)

Définition de service gRPC avec Protocol Buffers (protobuf)

```
1. // Définition du service en Protocol Buffers (.proto)
2. syntax = "proto3";
3.
4. package user;
5.
6. // Définition des messages
7. message GetUserRequest {
8.   int32 user_id = 1;
9.   bool include_posts = 2;
10. }
11.
12. message Post {
13.   int32 id = 1;
14.   string title = 2;
15.   string content = 3;
16.   string created_at = 4;
17. }
18.
19. message User {
20.   int32 id = 1;
```

EXEMPLES DE REQUÊTES : GRPC (PARTIE 1/2) [A]

Définition de service gRPC avec Protocol Buffers (protobuf) - Suite

```
21.   string name = 2;
22.   string email = 3;
23.   repeated Post posts = 4;
24. }
25.
26. message GetUserResponse {
27.   User user = 1;
28. }
29.
30. // Définition du service
31. service UserService {
32.   rpc GetUser(GetUserRequest) returns (GetUserResponse);
33.   rpc StreamUsers(StreamUsersRequest) returns (stream User);
34. }
```

EXEMPLES DE REQUÊTES : GRPC (PARTIE 2/2)

Client gRPC Python (python)

```python
# Utilisation du client gRPC en Python
import grpc
import user_pb2
import user_pb2_grpc

# Connexion au serveur gRPC
channel = grpc.insecure_channel('localhost:50051')
stub = user_pb2_grpc.UserServiceStub(channel)

# Création de la requête
request = user_pb2.GetUserRequest(
    user_id=123,
    include_posts=True
)

# Appel RPC (très rapide, binaire)
response = stub.GetUser(request)

print(f"User: {response.user.name}")
for post in response.user.posts:
```

EXEMPLES DE REQUÊTES : GRPC (PARTIE 2/2) [A]

Client gRPC Python (python) - Suite

```
21.     print(f"  - {post.title}")
```

CRITÈRES DE SÉLECTION : PERFORMANCE (PARTIE 1/3)

La performance est un critère déterminant dans le choix d'un paradigme d'API. Elle englobe plusieurs dimensions : latence, débit, taille des messages, utilisation des ressources serveur et réseau.

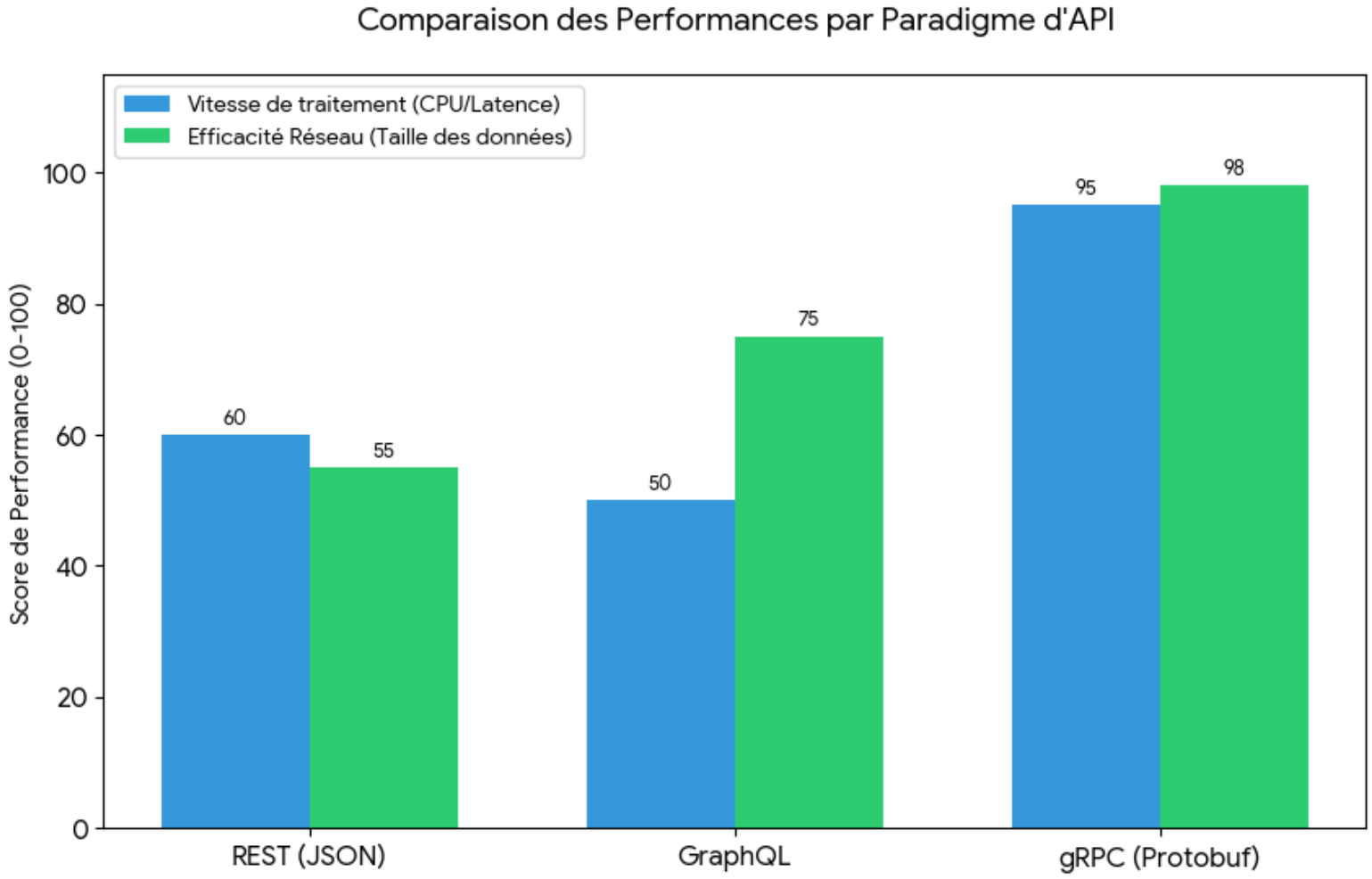
CRITÈRES DE SÉLECTION : PERFORMANCE (PARTIE 2/3)

gRPC : Performance maximale
- Sérialisation binaire Protocol Buffers (5-10x plus rapide que JSON)
- HTTP/2 avec multiplexing et compression des headers
- Streaming bidirectionnel natif
- Idéal pour : microservices, IoT, communications temps réel

REST : Performance équilibrée
- JSON lisible mais plus volumineux
- HTTP/1.1 standard (ou HTTP/2 possible)
- Caching HTTP natif très efficace
- Idéal pour : API publiques, intégrations tierces, applications web classiques

GraphQL : Performance variable
- Flexibilité au détriment de l'optimisation
- Problème N+1 côté serveur sans DataLoader
- Caching complexe (nécessite des stratégies personnalisées)
- Idéal pour : applications frontend complexes, agrégation de données multiples

CRITÈRES DE SÉLECTION : PERFORMANCE (PARTIE 3/3)



Note : gRPC surpasse REST et GraphQL en termes de latence et de bande passante grâce au format binaire et HTTP/2.

CRITÈRES DE SÉLECTION : COMPLEXITÉ D'IMPLÉMENTATION (PARTIE 1/2)

Complexité d'implémentation et d'adoption

Aspect	REST	GraphQL	gRPC
Courbe d'apprentissage	Faible - concepts HTTP standard	Moyenne - nouveau paradigme	Élevée - protobuf + concepts RPC
Setup initial	Simple - frameworks standards	Moyen - serveur GraphQL	Complexe - génération de code
Outillage	Excellent - mature et répandu	Bon - en croissance rapide	Bon - mais spécialisé
Documentation	OpenAPI/Swagger mature	Introspection native du schéma	Reflection API + protobuf
Debugging	Simple - outils HTTP standards	Moyen - requêtes complexes	Difficile - binaire
Compatibilité navigateur	Excellente - natif	Excellente - via HTTP	Limitée - nécessite gRPC-Web
Écosystème	Très mature	En croissance	Mature côté backend
Maintenance	Faible - stable	Moyenne - évolution schéma	Moyenne - gestion protobuf

CRITÈRES DE SÉLECTION : COMPLEXITÉ D'IMPLÉMENTATION (PARTIE 2/2)

La complexité d'implémentation doit être évaluée en fonction de l'équipe, du contexte et des contraintes temporelles. REST reste le choix le plus sûr pour démarrer rapidement avec des équipes juniors ou des contraintes de délai serrées.

CRITÈRES DE SÉLECTION : BESOINS MÉTIER (PARTIE 1/2)

Alignement avec les cas d'usage métier

CRITÈRES DE SÉLECTION : BESOINS MÉTIER (PARTIE 2/2)

Choisir REST quand :

- API publique avec de nombreux consommateurs externes
- Besoin de standards largement adoptés (OpenAPI)
- Caching HTTP crucial pour la performance
- Équipe avec peu d'expérience en API
- CRUD simple sur des ressources bien définies
- Exemples : API SaaS, intégrations partenaires, webhooks

Choisir GraphQL quand :

- Application frontend avec besoins de données variables
- Multiples clients (web, mobile) avec besoins différents
- Agrégation de données provenant de sources multiples
- Besoin de réduire le nombre de requêtes réseau
- Évolution rapide des besoins frontend
- Exemples : applications React/Vue complexes, dashboards personnalisables

CRITÈRES DE SÉLECTION : BESOINS MÉTIER (PARTIE 2/2)

Choisir gRPC quand :

- Architecture microservices avec communications internes intensives
- Performance critique et latence minimale requise
- Streaming temps réel bidirectionnel nécessaire
- Environnement polyglotte (multiples langages)
- Communication machine-to-machine
- Exemples : services backend, IoT, systèmes distribués haute performance

MATRICE DE DÉCISION : QUEL PARADIGME CHOISIR ?

Guide de sélection selon les priorités métier

Priorité principale	Paradigme recommandé	Justification
Simplicité et adoption rapide	REST	Standards matures, courbe d'apprentissage faible, outillage universel
Performance maximale	gRPC	Binaire, HTTP/2, streaming, latence minimale
Flexibilité client	GraphQL	Client contrôle les données, une seule endpoint, typage fort
Caching et CDN	REST	Caching HTTP natif, intégration CDN simple
API publique externe	REST	Compatibilité maximale, documentation standard (OpenAPI)
Microservices internes	gRPC	Performance, contrats stricts, génération de code
Application mobile/web riche	GraphQL	Réduction des requêtes, données sur mesure
Temps réel bidirectionnel	gRPC	Streaming natif performant
Évolutivité du schéma	GraphQL	Ajout de champs sans breaking changes
Interopérabilité maximale	REST	Standard universel, compatible tous navigateurs

TENDANCES ACTUELLES : API GATEWAY (PARTIE 1/3)

L'API Gateway est devenu un composant architectural essentiel dans les systèmes modernes. Il agit comme point d'entrée unique pour toutes les requêtes clients, centralisant la gestion de la sécurité, du routage, de la transformation et de l'orchestration des appels vers les microservices backend.

TENDANCES ACTUELLES : API GATEWAY (PARTIE 2/3)

Fonctionnalités principales d'un API Gateway :

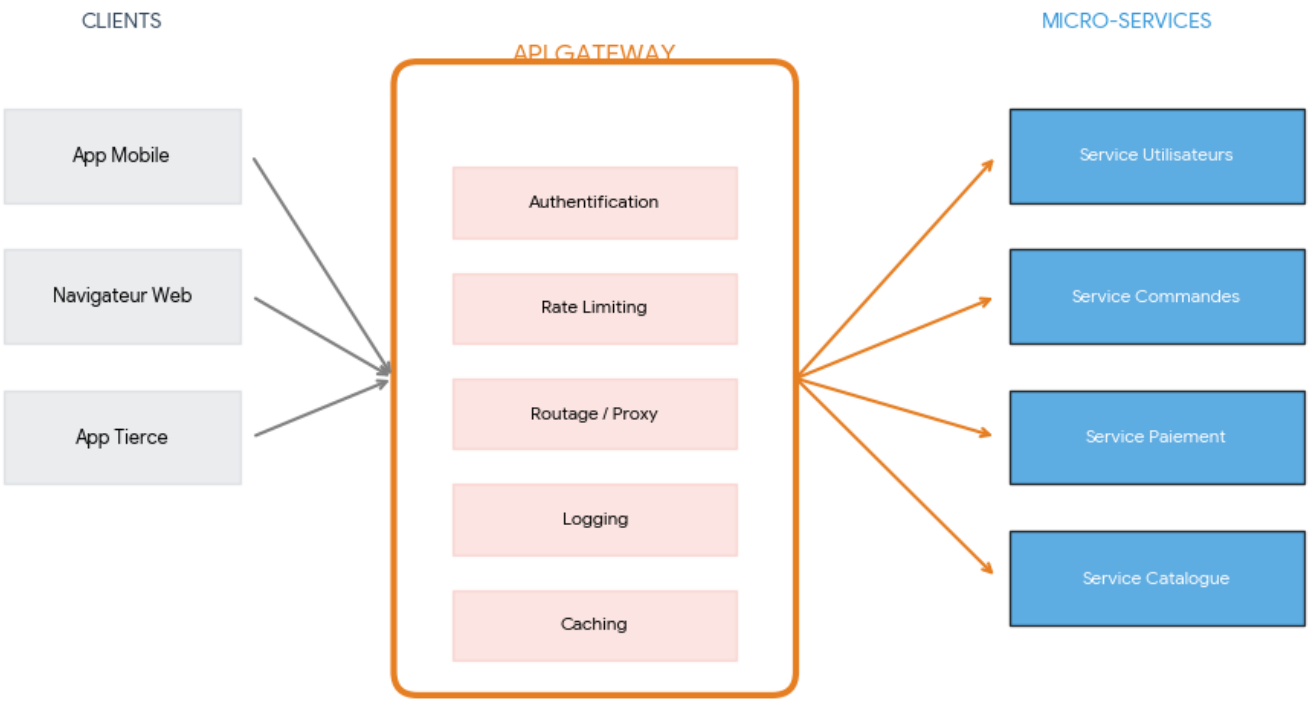
- Routage et load balancing vers les services backend
- Authentification et autorisation centralisées (OAuth2, JWT)
- Rate limiting et throttling pour protéger les services
- Transformation de protocoles (REST → gRPC, agrégation)
- Caching des réponses pour améliorer les performances
- Monitoring, logging et observabilité
- Gestion des versions d'API
- Circuit breaker et retry policies

Solutions populaires :

- Kong : Open source, extensible via plugins Lua
- AWS API Gateway : Solution cloud managée
- Azure API Management : Plateforme Microsoft complète
- Apigee (Google Cloud) : Enterprise-grade avec analytics
- Traefik : Cloud-native, intégration Kubernetes
- Envoy : Proxy haute performance, base d'Istio

TENDANCES ACTUELLES : API GATEWAY (PARTIE 3/3)

Tendances Actuelles : Architecture API Gateway pour Micro-services



L'API Gateway découple les clients des services backend.\nElle réduit la complexité pour le client et sécurise l'architecture.

CONFIGURATION D'UN API GATEWAY : EXEMPLE KONG (PARTIE 1/2)

Configuration déclarative Kong Gateway (yaml)

```yaml
# Configuration Kong via API déclarative (kong.yml)
_format_version: "3.0"

services:
  - name: user-service
    url: http://user-service:8080
    routes:
      - name: user-route
        paths:
          - /api/users
        methods:
          - GET
          - POST
    plugins:
      - name: rate-limiting
        config:
          minute: 100
          policy: local
      - name: jwt
        config:
```

CONFIGURATION D'UN API GATEWAY : EXEMPLE KONG (PARTIE 1/2) [A]

Configuration déclarative Kong Gateway (yaml) - Suite

```
21.           claims_to_verify:
22.             - exp
23.       - name: cors
24.         config:
25.           origins:
26.             - "*"
27.           methods:
28.             - GET
29.             - POST
30.           headers:
31.             - Authorization
32.             - Content-Type
33.
34.   - name: order-service
35.     url: http://order-service:8080
36.     routes:
37.       - name: order-route
38.         paths:
39.           - /api/orders
40.     plugins:
```

CONFIGURATION D'UN API GATEWAY : EXEMPLE KONG (PARTIE 1/2) [B]

Configuration déclarative Kong Gateway (yaml) - Suite

```
41.       - name: rate-limiting
42.         config:
43.           minute: 50
44.       - name: request-transformer
45.         config:
46.           add:
47.             headers:
48.               - X-Service-Name:order-service
```

CONFIGURATION D'UN API GATEWAY : EXEMPLE KONG (PARTIE 2/2)

Cette configuration définit deux services avec leurs routes et plugins associés. Le rate limiting protège contre les abus, JWT assure l'authentification, et CORS permet les appels cross-origin. Kong applique ces politiques de manière centralisée sans modification du code des microservices.

RATE LIMITING ET THROTTLING (PARTIE 1/3)

Le rate limiting est crucial pour protéger les API contre les abus, garantir une qualité de service équitable entre utilisateurs, et prévenir les attaques DDoS. Il limite le nombre de requêtes qu'un client peut effectuer dans une période donnée.

RATE LIMITING ET THROTTLING (PARTIE 2/3)

Stratégies de rate limiting :

- Fixed Window : limite par fenêtre de temps fixe (simple mais edge cases)
- Sliding Window : fenêtre glissante pour distribution uniforme
- Token Bucket : accumulation de tokens, permet les bursts contrôlés
- Leaky Bucket : débit constant, lisse les pics de trafic

Niveaux de granularité :

- Par IP : protection basique contre abus
- Par utilisateur/API key : limites personnalisées selon le plan tarifaire
- Par endpoint : protection des opérations coûteuses
- Global : protection de l'infrastructure entière

Communication au client :

- Headers HTTP standards : X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset
- Code 429 Too Many Requests avec Retry-After header
- Documentation claire des limites dans l'API

RATE LIMITING ET THROTTLING (PARTIE 3/3)

(python)

```python
# Implémentation rate limiting avec Redis en Python (Flask)
from flask import Flask, request, jsonify
from redis import Redis
import time

app = Flask(__name__)
redis_client = Redis(host='localhost', port=6379, decode_responses=True)

def check_rate_limit(user_id, limit=100, window=60):
    """
    Token bucket algorithm avec Redis
    limit: nombre max de requêtes
    window: fenêtre en secondes
    """
    key = f"rate_limit:{user_id}"
    current_time = int(time.time())

    # Pipeline Redis pour atomicité
    pipe = redis_client.pipeline()
    pipe.zadd(key, {current_time: current_time})
```

RATE LIMITING ET THROTTLING (PARTIE 3/3) [A]

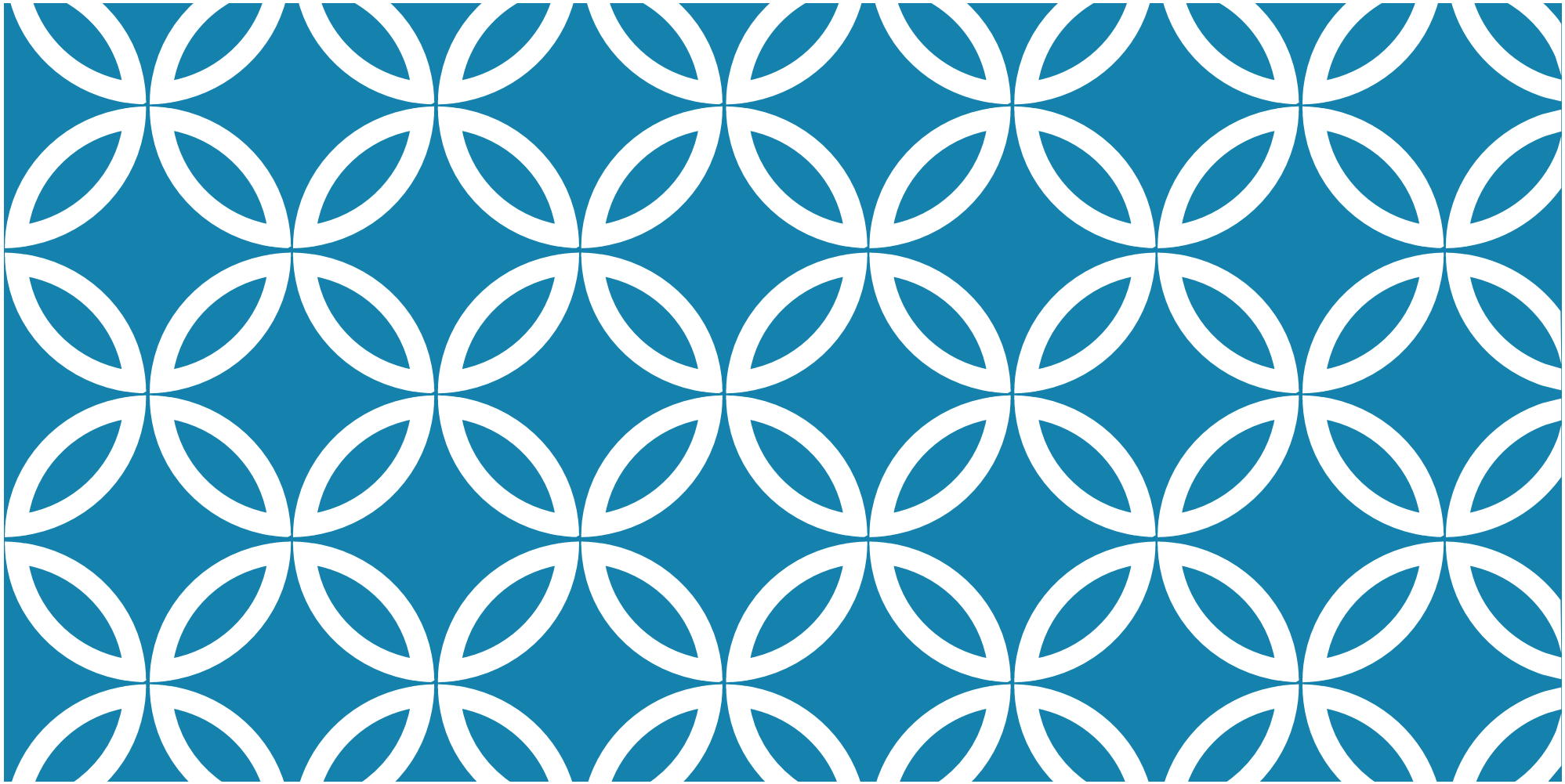
(python) - Suite

```
21.     pipe.zremrangebyscore(key, 0, current_time - window)
22.     pipe.zcard(key)
23.     pipe.expire(key, window)
24.     results = pipe.execute()
25.
26.     request_count = results[2]
27.
28.     return {
29.         'allowed': request_count <= limit,
30.         'remaining': max(0, limit - request_count),
31.         'reset': current_time + window
32.     }
33.
34. @app.route('/api/data')
35. def get_data():
36.     user_id = request.headers.get('X-User-ID', 'anonymous')
37.
38.     rate_limit = check_rate_limit(user_id, limit=100, window=60)
39.
40.     # Ajout des headers de rate limiting
```

RATE LIMITING ET THROTTLING (PARTIE 3/3) [B]

(python) - Suite

```
41.     headers = {
42.         'X-RateLimit-Limit': '100',
43.         'X-RateLimit-Remaining': str(rate_limit['remaining']),
44.         'X-RateLimit-Reset': str(rate_limit['reset'])
45.     }
46.
47.     if not rate_limit['allowed']:
48.         return jsonify({
49.             'error': 'Rate limit exceeded',
50.             'retry_after': rate_limit['reset'] - int(time.time())
51.         }), 429, headers
52.
53.     return jsonify({'data': 'Your data here'}), 200, headers
```



MERCI DE VOTRE ATTENTION